

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ «МИФИ»

КАФЕДРА ФИНАНСОВОГО МОНИТОРИНГА

УДК 004.4:004.85:004.056.55

«УТВЕРЖДАЮ»
зам. зав. кафедрой,
канд. экон. наук
_____/ А.Н. Норкина
(подпись)
« ____ » _____ 2024 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

ПАРАХАТОВ КАНАТ САГЫНДЫКОВИЧ

«РАЗРАБОТКА МОБИЛЬНОГО ПРИЛОЖЕНИЯ ДЛЯ ANDROID СМАРТФОНОВ,
ОБЕСПЕЧИВАЮЩЕЕ ИНТЕРАКТИВНОЕ ОБУЧЕНИЕ АЛГОРИТМАМ КРИПТОГРА-
ФИЧЕСКОГО ШИФРОВАНИЯ»

Специальность 10.04.01 Информационная безопасность

Направление подготовки: "Информационно-аналитическое обеспечение финансового мо-
ниторинга"

Руководитель проекта старший преподаватель кафедры 75 _____ Рычков В.А.
(должность, ученая степень) (подпись)

Москва 2024

Студент-дипломник

(подпись, дата)

К.С. Парахатов

Нормоконтроль

(подпись, дата)

Д.С. Павлов

Рецензент

(подпись, дата)

К.К. Сеитназаров

Зам. зав. кафедрой

(подпись, дата)

А.Н. Норкина

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ
«МИФИ»

КАФЕДРА ФИНАНСОВОГО МОНИТОРИНГА

Направление подготовки 10.04.01 «Информационная безопасность»
Магистерская программа «Информационно-аналитическое
обеспечение финансового мониторинга»

«Утверждаю»
зам. зав. кафедрой
_____ / А.Н. Норкина
(подпись)
«__» _____ 2024 г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы магистра

Фамилия, имя, отчество студента	Парахатов Канат Сагындыкович
Тема ВКР	Разработка мобильного приложения для Android смартфонов, обеспечивающее интерактивное обучение алгоритмам криптографического шифрования
Место выполнения ВКР	Национальный исследовательский ядерный университет «МИФИ»
Руководитель ВКР (ФИО, должность, место работы)	Рычков Вадим Александрович, старший преподаватель, НИЯУ «МИФИ»
Цель работы	Целью работы является разработка мобильного приложения для ОС Android, предоставляющего интерактивный метод обучения криптографическим алгоритмам шифрования.

1. Исходные данные

Интерактивное приложение для Android по обучению криптографии, целью которой является разработка мобильного приложения, которое предложит пользователям интерактивный способ укрепления практических знаний или изучения методов криптографического шифрования.

2. Экономическое обоснование

Существующие методы обучения криптографии часто сложны. Разрабатываемое Android - приложение призвано решить эту проблему, предлагая доступный, удобный и интерактивный способ изучения алгоритмов шифрования.

Приложение имеет следующие достоинства:

- Доступность: Приложение будет бесплатным или иметь низкую стоимость, что сделает его доступным для широкой аудитории;
- Удобство: Пользователи смогут обучаться в любое удобное время и в любом месте, имея под рукой лишь смартфон;
- Интерактивность: Интерактивные элементы сделают обучение более эффективным, чем традиционные методы;
- Актуальность: Контент приложения можно легко обновлять, добавляя новые алгоритмы и информацию по мере развития технологий.

Возможный экономический эффект:

- Потенциал монетизации: Приложение может быть монетизировано с помощью рекламных баннеров или пожертвований;
- Снижение затрат на обучение: Приложение может стать доступной альтернативой дорогим онлайн-курсам и специализированным программам.

Социальный эффект:

- Повышение уровня цифровой грамотности: Приложение будет способствовать повышению уровня знаний о криптографии и информационной безопасности среди пользователей;
- Создание более безопасной информационной среды: Чем больше людей будут знать о методах защиты информации, тем более защищенной будет информационная среда в целом.

1. Научно-исследовательская часть

Обзор и анализ существующих решений: анализ и изучение существующих мобильных приложений, которые предлагают обучение криптографии. Оценка их функционала, интерфейса, методов обучения и ограничений. Изучение и описание симметричных и асимметричных алгоритмов шифрования.

Технологии разработки мобильных приложений: архитектура платформы, основы разработки под Android, включая Android SDK и Android Studio, и Kotlin для разработки мобильных приложений.

2. Оформление ВКР

2.1. Расчетно-пояснительная записка на 100 листах формата А4.

2.2. Перечень графического материала (схемы, чертежи и т.п.)

Рисунки, таблицы _____

КАЛЕНДАРНЫЙ ПЛАН
выполнения выпускной квалификационной работы магистра

№ п/п	Наименование этапов работы	Срок начала выполнения этапа	Срок завершения выполнения этапа
1.	Глава 1. (Анализ основных понятий и алгоритмов шифрования)	16.01.2024	12.02.2024
2.	Глава 2. (Моделирование структуры приложения)	13.02.2024	05.04.2024
3.	Глава 3. (Процесса разработки мобильного приложения)	06.04.2024	28.05.2024

Дата выдачи задания «16» января 2024 г.

В соответствии с учебным планом выпускную квалификационную работу выполнить в полном объеме в срок до «05» июня 2024 г.

Руководитель ВКР _____
 (Подпись, дата)

В.А. Рычков
 (И.О. Фамилия)

Задание принял к исполнению _____
 (Подпись, дата)

К.С.Парахатов
 (И.О. Фамилия)

АННОТАЦИЯ К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ МАГИСТРА

Выпускная квалификационная работа магистра на тему «Разработка мобильного приложения для Android смартфонов, обеспечивающее интерактивное обучение алгоритмам криптографического шифрования» состоит из 54 страниц, 1 таблиц, 43 рисунков, использованных источников - 33.

В связи с растущей цифровизации и важности кибербезопасности, обеспечивать доступность и понятность криптографических алгоритмов становится важным элементом личной цифровой безопасности. Мобильное приложение предоставит возможность обучения и укреплению знаниям в области криптографии в удобной форме для широкого круга пользователей, от студентов до специалистов в области информационной безопасности.

Объектом исследования является процесс обучения пользователей методам шифрования с помощью мобильного приложения. Это включает в себя как передачу теоретических знаний о криптографических алгоритмах, так и их практическое применение через интерактивный подход.

Предметом исследования являются интерактивное обучение алгоритмам шифрования, реализованные в данном мобильном приложении. В частности, исследование касается теоретических аспектов шифрования и представления пошагового выполнения алгоритмов на практике.

Цель исследования: Целью исследования является создание инструмента для изучения и укрепления знания о криптографических алгоритмах с помощью мобильного приложения, которое сочетает теоретическую часть и практическое применение через интерактивный подход и визуализацию.

Задачи исследования:

1. Анализ литературы и существующих решений: Изучить современные подходы к обучению криптографии, в том числе используемые методы и средства.
2. Разработка теоретической части:
 - Создать подробные учебные материалы по каждому алгоритму шифрования (Шифр Цезаря, Лозунговый шифр, AES, DES, Магма, RSA, Эль-Гамаль, Диффи-Хеллман).
3. Разработка практической части:
 - Создать интерактивные подходы, позволяющие пользователям вводить свои данные и видеть пошаговое выполнение алгоритмов;
 - Разработать интерфейс, который будет интуитивно понятен и удобен для пользователя.
4. Тестирование и отладка:
 - Провести тестирование корректности работы каждого алгоритма;
 - Обеспечить совместимость приложения с различными версиями Android и устройствами.

Практическая значимость исследования заключается в создании образовательного инструмента, который может быть использован:

- Самостоятельными учениками и энтузиастами для изучения криптографических алгоритмов;
- Разработчиками программного обеспечения для понимания и реализации криптографических методов в своих проектах.

Приложение поможет пользователям лучше понять, как работают различные алгоритмы шифрования, предоставляя им интерактивные средства для изучения и практического применения этих знаний.

Для достижения целей и задач применяются методы анализа, моделирования и разработки мобильного приложения, а также осуществляется тестирование и сравнительный анализ с аналогичными программами.

Научная новизна исследования заключается в создании уникального инструмента для обучения криптографии, который объединяет теоретическое и практическое обучение в одном мобильном приложении. Это приложение позволяет пользователям:

- Глубже понять принципы работы криптографических алгоритмов через визуализации и пошаговые объяснения;
- Практически применять полученные знания, вводя свои данные и получив результат пошагового шифрования введенных данных;
- Усовершенствовать и закреплять свои познания через интерактивные подходы, что является более эффективным по сравнению с традиционными методами обучения.

Тема выпускной квалификационной работы соответствует требованиям современных образовательных стандартов, так как ориентирована на разработку современного образовательного продукта, который способствует эффективному освоению алгоритмов шифрования в процессе обучения и соответствует паспорту специальности 10.04.01 «Информационная безопасность».

СОДЕРЖАНИЕ РАБОТЫ

Во введении - обосновывается выбор темы исследования, а также ее актуальность в контексте современных требований к информационной безопасности. Определяются цели работы и задачи, которые нужно выполнить для их достижения.

В первой главе проводится анализ понятий криптографии и алгоритмов шифрования. Изучаются как классические, так и современные методы шифрования. Проводится обзор существующих образовательных ресурсов.

Вторая глава посвящена моделированию структуры приложения. Здесь описываются выбранные инструменты и технологии для разработки, процесс моделирования структуры приложения.

В третьей главе представлено описание процесса разработки мобильного приложения. Рассматривается установка и настройка необходимых программных средств, разработка функционала и интерфейса приложения, а также использование языка программирования Kotlin, сред разработки Android Studio и IntelliJ IDEA.

Заключение сводит все результаты исследования и делает основные выводы практической значимости. Оценивается эффективность созданного приложения для обучения алгоритмам шифрования и проводится сопоставление достигнутых результатов с поставленными целями и задачами работы.

ANNOTATION TO THE MASTER'S GRADUATE QUALIFICATION WORK

Master's graduate qualification paper on “Development of mobile application for Android smartphones, providing inter-active learning of cryptographic encryption algorithms” consists of 54 pages, 1 tables, 43 figures, used sources - 33.

Due to increasing digitalization and the importance of cybersecurity, ensuring that cryptographic algorithms are accessible and understandable is becoming an important element of personal digital security. The mobile application will provide an opportunity to learn and strengthen knowledge in the field of cryptography in a convenient form for a wide range of users, from students to specialists in the field of information security.

The object of the study is the process of teaching user's encryption techniques using a mobile application. This includes both the transfer of theoretical knowledge about cryptographic algorithms and their practical application through an interactive approach.

The subject of the study is the interactive training of encryption algorithms implemented in this mobile application. In particular, the study concerns theoretical aspects of encryption and the presentation of step-by-step execution of algorithms in practice.

Research aim: The aim of the study is to create a tool for learning and reinforcing knowledge about cryptographic algorithms through a mobile application that combines theory and practical application through an interactive approach and visualization.

Research Objectives:

1. Analyze the literature and existing solutions: Examine current approaches to teaching cryptography, including the methods and tools used.

2. Development of the theoretical part:

- Create detailed training materials for each encryption algorithm (Caesar Cipher, Playfair Cipher, AES, DES, Magma, RSA, El - Gamal, Diffie - Hellman).

3. Development of the practical part:

- Create interactive approaches that allow users to input their data and see the step-by-step execution of the algorithms;
- Develop an interface that is intuitive and user-friendly.

4. Testing and debugging:

- Conduct correctness testing of each algorithm;
- Ensure compatibility of the application with different Android versions and devices.

The practical significance of the study lies in the creation of an educational tool that can be used by:

- By self-students and enthusiasts to learn cryptographic algorithms;
- By software developers to understand and implement cryptographic techniques in their projects.

The application will help users to better understand how different encryption algorithms work by providing them with interactive tools to learn and practice this knowledge.

To achieve the goals and objectives, methods of analysis, modeling and development of the mobile application are used, as well as testing and comparative analysis with similar programs.

The scientific novelty of the research lies in the creation of a unique tool for learning cryptography, which combines theoretical and practical training in one mobile application. This app allows users to:

- Deepen their understanding of the principles of cryptographic algorithms through visualizations and step-by-step explanations;
- Practically apply what they have learned by entering their data and getting the result of step-by-step encryption of the entered data;
- To improve and consolidate their knowledge through an interactive approach, which is more effective than traditional teaching methods.

The topic of the final qualification work corresponds to the requirements of modern educational standards, as it is focused on the development of a modern educational product that promotes the effective mastering of encryption algorithms in the learning process and corresponds to the passport of the specialty 10.04.01 “Information Security”.

WORK CONTENT

In the introduction - the choice of the topic of the study is justified, as well as its relevance in the context of modern requirements for information security. The objectives of the work and the tasks to be performed to achieve them are defined.

The first chapter analyzes the concepts of cryptography and encryption algorithms. Both classical and modern encryption methods are studied. The existing educational resources are reviewed.

The second chapter is devoted to modeling the structure of the application. It describes the selected tools and technologies for development and the process of modeling the structure of the application.

The third chapter describes the process of developing a mobile application. The installation and configuration of the necessary software, development of the application functionality and interface, as well as the use of the Kotlin program-

ming language, Android Studio and IntelliJ IDEA development environments are considered.

The conclusion summarizes all the results of the study and draws the main conclusions of practical significance. It evaluates the effectiveness of the created application for teaching encryption algorithms and compares the achieved results with the goals and objectives of the work.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	16
1 Анализ основных понятий и алгоритмов шифрования.....	22
1.1 Обзор основных понятий криптографии	22
1.2 Анализ классических и современных алгоритмов шифрования.....	23
1.3 Обзор имеющихся образовательных ресурсов	29
1.4 Целевая аудитория разрабатываемого приложения.....	32
1.5 Сравнение проанализированных мобильных приложений	33
2 Моделирование структуры приложения.....	36
2.1 Описание выбранных инструментов и технологий для разработки.....	36
2.2 Процесс моделирования структуры приложения	41
3 Процесса разработки мобильного приложения и инструкция пользования приложением.....	43
3.2 Установка и настройка необходимых программных средств	43
3.3 Алгоритмы шифрования дешифрования. Написание и тестирование алгоритмов	49
3.4 Разработка интерфейса приложения.....	53
3.5 Инструкция пользования приложением	56
ЗАКЛЮЧЕНИЕ	68
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	70
ПРИЛОЖЕНИЕ А	74
ПРИЛОЖЕНИЕ Б.....	76
ПРИЛОЖЕНИЕ В	78
ПРИЛОЖЕНИЕ Г.....	89
ПРИЛОЖЕНИЕ Д	95
ПРИЛОЖЕНИЕ Ж	98
ПРИЛОЖЕНИЕ И	100
ПРИЛОЖЕНИЕ К	102

ВВЕДЕНИЕ

Смартфоны - это устройства, объединяющий в себе функции телефона, компьютера, фотоаппарата и многих других устройств. Они стали неотъемлемой частью повседневной жизни многих людей. С помощью смартфонов можно звонить, отправлять сообщения, смотреть видео, слушать музыку, играть в игры, пользоваться интернетом, учиться, делать фотографии и многое другое. Благодаря своей компактности и мобильности смартфоны всегда под рукой, помогают людям оставаться на связи, весело проводить время за игрой и решать различные задачи в любое время и в любом месте.

В 2024 году более 4.88 млрд человек владеют смартфонами. Это означает, что 60.42% населения Земли используют смартфоны. За последние десять лет количество пользователей смартфонов увеличилось более чем в четыре раза, за последние пять лет - это число выросло вдвое [7]. Статистика количества владельцев смартфонов в мире представлена на рисунке 1.



Рисунок 1 – Количество владельцев смартфонов в мире и прогноз на следующие годы.
(источник: inclient.ru/smartphone-stats/#lwptoc)

В 2024 году в мире насчитывается 7.21 млрд смартфонов. Это на 28.98% больше, чем было в 2019 году, когда общее количество устройств со-

ставило 5.59 млрд. За последние десять лет число смартфонов в мире выросло в три раза [7]. Статистика количества смартфонов в мире представлена на рисунке 2.



Рисунок 2 – Количество смартфонов в мире и прогноз на следующие годы.
(источник: inclient.ru/smartphone-stats/#lwptoc)

По данным Statista, в 2024 году 104,31 млн россиян используют смартфоны. В 2022 году смартфонами пользовались 106,44 млн россиян [8].

Согласно отчету Минцифры, число активных абонентов мобильной связи в конце 2022 года составило 245,3 млн. Этот показатель вырос на 7,6 млн с 2010 года [9]. В 2020 году, по данным ТАСС, 70% россиян использовали смартфон для выхода в интернет [10].

С 2024 по 2029 год количество пользователей смартфонов в России будет постоянно расти. Согласно прогнозам, за этот период число пользователей вырастет на 17,4 млн человек, что составит 16,68% от общего количества пользователей в 2024 году [8].

Более половины нашего времени в сети мы проводим на мобильных телефонах. Добавление беспроводных подключений к нашим портативным устройствам сформировало современную цифровую среду как для аппарат-

ного, так и для программного обеспечения, одновременно способствуя цифровизации на развивающихся рынках. По оценкам, в середине 2023 года почти 96% цифрового населения планеты использовали мобильные устройства для подключения к Интернету [11].

По данным radar.yandex.ru, в России с 15 января 2024 года по 17 января 2024 года 69,12% пользователей использовали смартфоны, 29,5% использовали компьютер и 1,38% планшеты [12]. Статистику соотношения использования смартфона, компьютера и планшета по России за полный год и по 15-17 января 2024 года, см. на рисунке 3.

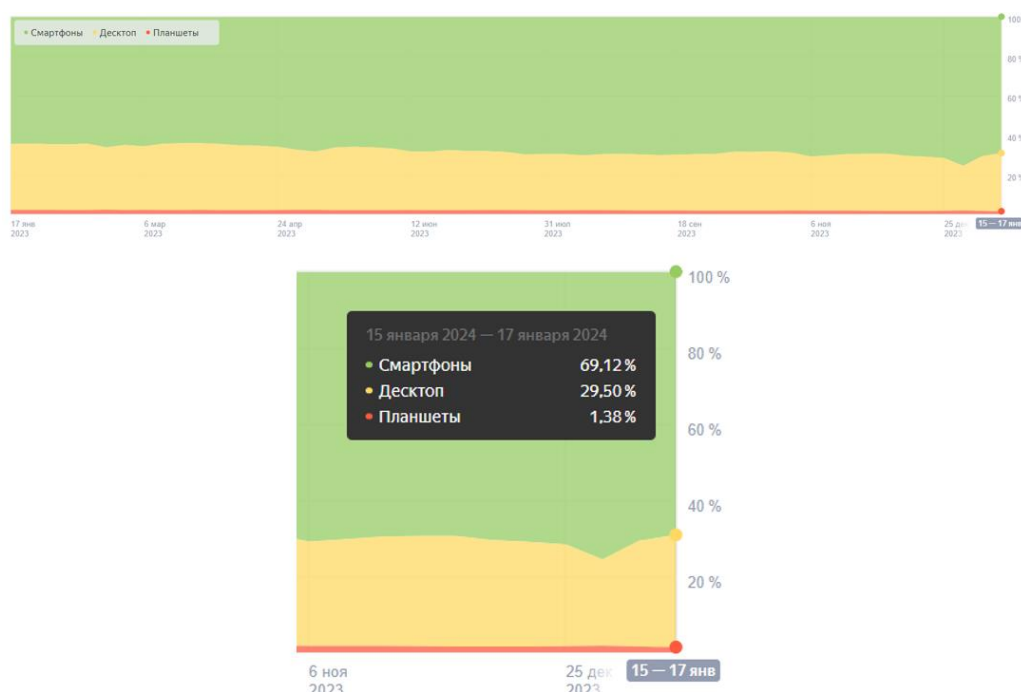


Рисунок 3 – Соотношение пользователей смартфона, компьютера и планшета по России за период 17.01.2023-17.01.2024 гг. и за период 15-17.01.2024 г.
(источник: radar.yandex.ru)

Учитывая результаты данных статистик заметно, что большинство пользователей в качестве основного устройства используют смартфоны нежели компьютеры и планшеты.

81.81% пользователей смартфонов являются владельцами смартфонов с операционной системой Android, 18.17% являются владельцами смартфонов с операционной системой iOS, остальные 0.01% остальные операционные си-

стемы на период 15-17 января 2024 г. [12]. Статистику соотношения операционных систем смартфонов Android, iOS и остальных ОС по России, см. на рисунке 4.

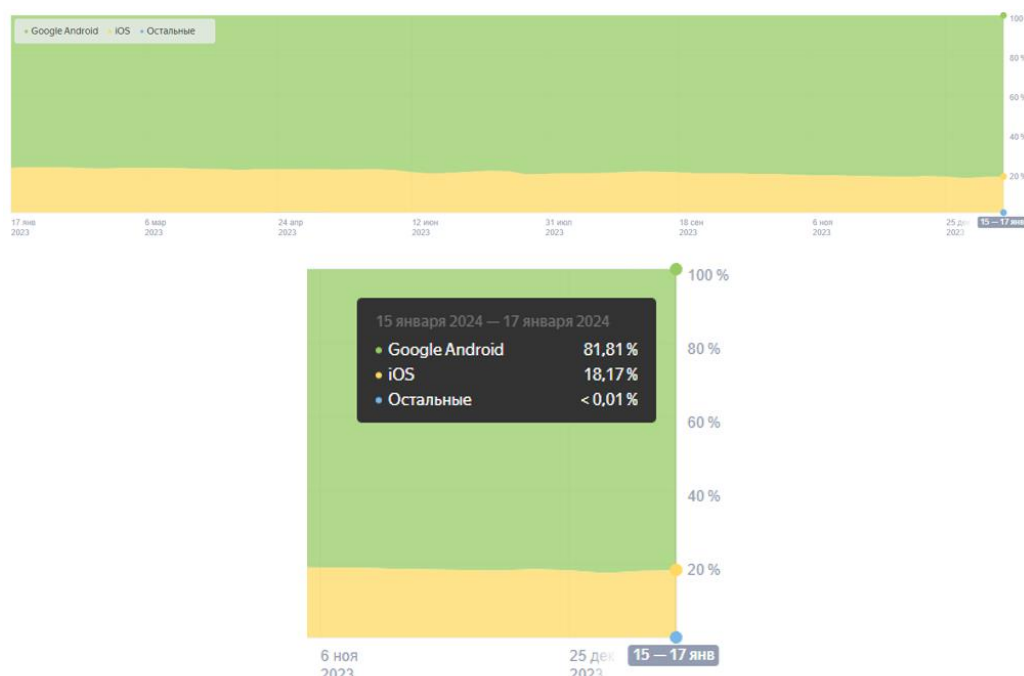


Рисунок 4 – Соотношение операционных систем смартфона, компьютера и планшета за период 17.01.2023-17.01.2024 гг. и за период 15-17 января 2024 г.
(источник: radar.yandex.ru)

Учитывая результаты вышеперечисленных статистик было решено разработать мобильное приложения для Android смартфонов с интерактивным подходом изучения алгоритмов криптографического шифрования, т.к. это является наиболее удобным для большинства интересующихся и будущих пользователей данного приложения.

В наше время использование сайтов для изучения криптографическим методам шифрования с мобильного устройства вызывает неудобства. Для использования необходимо запоминать адрес нужного сайта или выполнять поиск в поисковых системах. Далее необходимо найти на сайте нужный раздел среди множества ссылок и большого текста.

Технологии развиваются семимильными шагами. Большинство сайтов предоставляющие различные услуги сперва предлагают зарегистрироваться и затем выполнить необходимую функцию. В связи с этим всё больше информации как логины, пароли, данные банковских и другие должны быть надёжно защищены. Все эти данные надёжно и в секрете в зашифрованном виде хранятся, используя различные алгоритмы криптографического шифрования. Самые распространённые алгоритмы криптографического шифрования это AES, Магма, 3DES, RSA, Схема Эль-Гамала и другие. Они широко используются в информационной безопасности.

Основной задачей программного обеспечения является быстрое обучения алгоритмам шифрования. Входные данные задаются пользователем и результат будет выводом вычисления пошагового шифрования введённых данных, это облегчает в понимании как работает тот или иной алгоритм криптографического шифрования.

В этом приложении реализованы классические алгоритмы шифрования, симметричные, асимметричные алгоритмы шифрования, а также протокол обмена ключами:

- классические алгоритмы шифрования, такие как «Шифр Цезаря» и «Лозунговый шифр»;
- симметричные алгоритмы шифрования, такие как «AES», «DES» и «Магма (ГОСТ 34.12-2018)»;
- асимметричные алгоритмы шифрования, такие как «RSA», «Схема Эль-Гамала»;
- протокол обмена ключами, как «Протокол обмена ключами Диффи-Хеллмана».

Цель исследования - разработать мобильное приложение для Android смартфонов, которое поможет пользователям понять и усвоить алгоритмы шифрования. Чтобы достичь этой цели, необходимо выполнить следующие задачи:

- Изучить и описать основные понятия и алгоритмы шифрования;
- Проанализировать существующие образовательные ресурсы и определить, как лучше всего подать материал для пользователей;
- Создать структуру приложения и реализовать его с помощью Kotlin и Android Studio.

Предмет исследования - это разработка мобильного приложения для Android смартфонов для обучения алгоритмам криптографического шифрования.

1 Анализ основных понятий и алгоритмов шифрования

1.1 Обзор основных понятий криптографии

Криптография - это наука о методах защиты информации путем преобразования ее в форму, недоступную для неавторизованных пользователей.

Шифрование - это процесс преобразования информации (открытого текста) в зашифрованную форму (шифротекст) с помощью алгоритма и ключа. После шифрования информации современными алгоритмами шифрования, шифротекст выглядит как случайная последовательность символов и не может быть прочитан без ключа.

Дешифрование - это обратный процесс, при котором шифротекст преобразуется обратно в открытый текст с использованием ключа.

Ключ - это секретная информация, используемая для шифрования и дешифрования данных. Без правильного ключа невозможно прочитать зашифрованное сообщение. Ключи могут быть симметричными (одинаковые для шифрования и дешифрования) или асимметричными (разные ключи для шифрования и дешифрования).

Все методы преобразования информации с целью защиты от несанкционированного доступа делятся на две большие группы: методы шифрования с закрытым ключом и методы шифрования с открытым ключом.

Для шифрования и расшифровывания данных в методах с закрытым ключом используется один и тот же ключ, который обе стороны стараются хранить в секрете от противника. Например, Шифр Цезаря, Лозунговый шифр, AES, DES и Магма.

В шифровании с открытым ключом (асимметричное шифрование) для шифрования и расшифровывания данных используются два разных ключа. Например, RSA, Схема Эль-Гамала и Протокол обмена ключами Диффи-Хеллмана.

Цифровая подпись - это криптографический механизм, который позволяет подтвердить подлинность и целостность сообщения или документа. Она создается с использованием закрытого ключа отправителя и проверяется с помощью открытого ключа.

Хэш-функция - это алгоритм, который преобразует входные данные произвольной длины в фиксированную строку (хэш). Данная функция является односторонней функцией, т.е. его вычислить легко, но обратную значению хэш-функции вычислить невозможно. Пока это ещё не доказано математическим методом. Хэш-функции широко используются для проверки целостности данных и хранения паролей. Пример: SHA-256 (Secure Hash Algorithm).

Протоколы обмена ключами - это методы, которые позволяют двум сторонам безопасно обмениваться ключами шифрования. Пример: протокол Диффи-Хеллмана, который позволяет двум сторонам установить общий секретный ключ, используя открытые каналы связи.

1.2 Анализ классических и современных алгоритмов шифрования

В этой главе будут рассмотрены основные алгоритмы шифрования, начиная с классических методов, которые использовались в прошлом, и заканчивая современными алгоритмами, широко применяемыми сегодня.

Классические алгоритмы шифрования:

1. Шифр Цезаря

- Описание: Один из самых древних и простых методов шифрования, при котором каждая буква в тексте смещается на фиксированное число позиций в алфавите. Например, при сдвиге на 3 буква А становится Г, В - Е и так далее, см. рисунок 5;
- Преимущества: Простота реализации и понимания.
- Недостатки: Легко поддается взлому методом частотного анализа и перебора всех возможных ключей.



Рисунок 5 – Шифр Цезаря
(источник: Яндекс картинки)

2. Лозунговый шифр (Playfair cipher)

- Описание: шифр, также известный как шифр с ключевым словом, использует ключевое слово (лозунг) для построения таблицы шифрозамен. Этот метод шифрования был популярен из-за своей простоты и удобства использования. Выбирается ключевое слово (лозунг). Желательно, чтобы оно включало конечные буквы алфавита. Создается строка, в которую сначала записываются буквы ключевого слова, пропуская повторяющиеся буквы. Затем добавляются остальные буквы алфавита в алфавитном порядке, исключая те, которые уже присутствуют в ключевом слове. Для примера ключ «АЛЛЕЯ», см. рисунок 6;
- Преимущества: Сложнее взломать, чем простые моноалфавитные шифры.
- Недостатки: Уязвим к методам криптоанализа, таким как частотный анализ на биграммах.

А	Б	В	Г	Д	Е	Ё	Ж	З	И	Й	К	Л	М	Н	О
А	Л	Е	Я	Б	В	Г	Д	Ё	Ж	З	И	Й	К	М	Н

П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю

Рисунок 6 – Таблицы замен для Лозунгового шифра

Современные алгоритмы шифрования:

3. AES (Advanced Encryption Standard)

- Описание: Современный симметричный алгоритм шифрования, использующий ключи длиной 128, 192 или 256 бит. Разработан на основе сети Фейстеля. AES оперирует данными в блоках по 128 бит и выполняет несколько раундов замены и перестановки байтов, см. рисунок 7;
- Преимущества: Высокая безопасность и производительность, стандарт для шифрования данных;
- Недостатки: Сложность реализации и необходимость в управлении ключами.

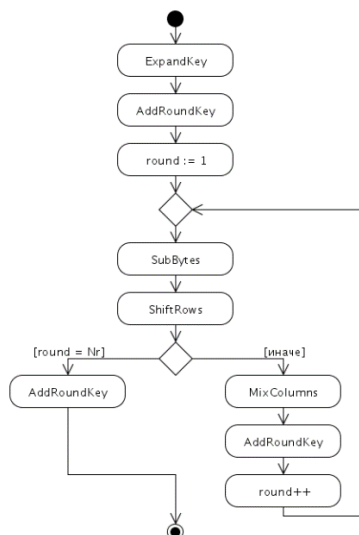


Рисунок 7 – Алгоритм шифрования AES на основе сети Фейстеля
(источник: Яндекс картинки)

4. DES (Data Encryption Standard)

- Описание: Симметричный алгоритм блочного шифрования, использующий 56-битный ключ и оперирующий данными в блоках по 64 бита. Выполняет 16 раундов шифрования. Разработан на основе сети Фейстеля, см. рисунок 8;
- Преимущества: Легкость реализации и понимания;

- Недостатки: Недостаточная длина ключа, что делает его уязвимым для атак методом полного перебора.

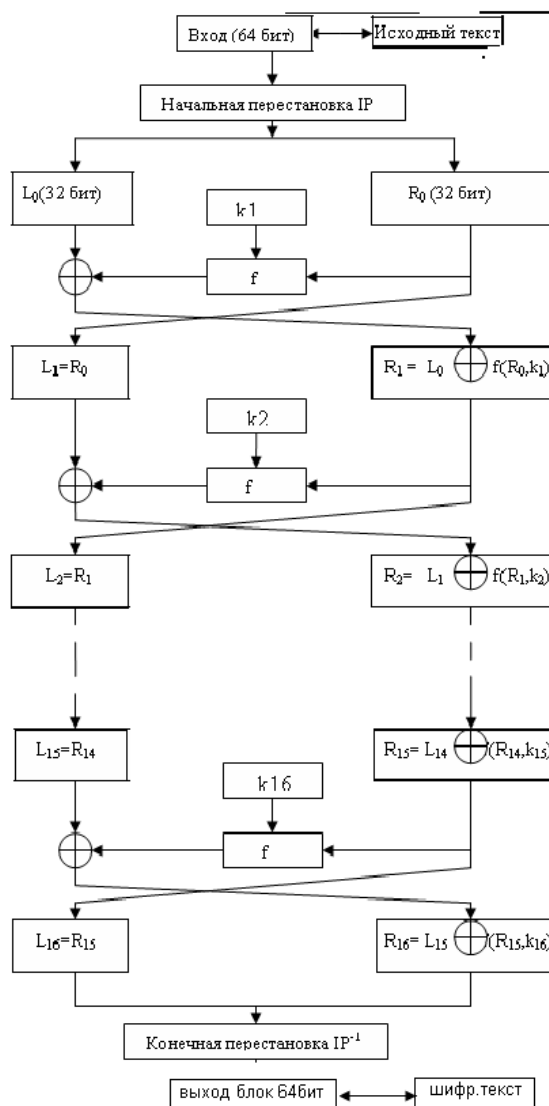


Рисунок 8 - Алгоритм шифрования DES на основе сети Фейстеля
(источник: Яндекс картинки)

5. Магма (ГОСТ 34.12-2018)

- Описание: Российский стандарт блочного шифрования, использующий ключ длиной 256 бит и 32 раунда шифрования. Разработан на основе сети Фейстеля, см. рисунок 9;
- Преимущества: Высокая надежность при правильной реализации;

- Недостатки: Меньшая распространенность и поддержка в сравнении с AES.

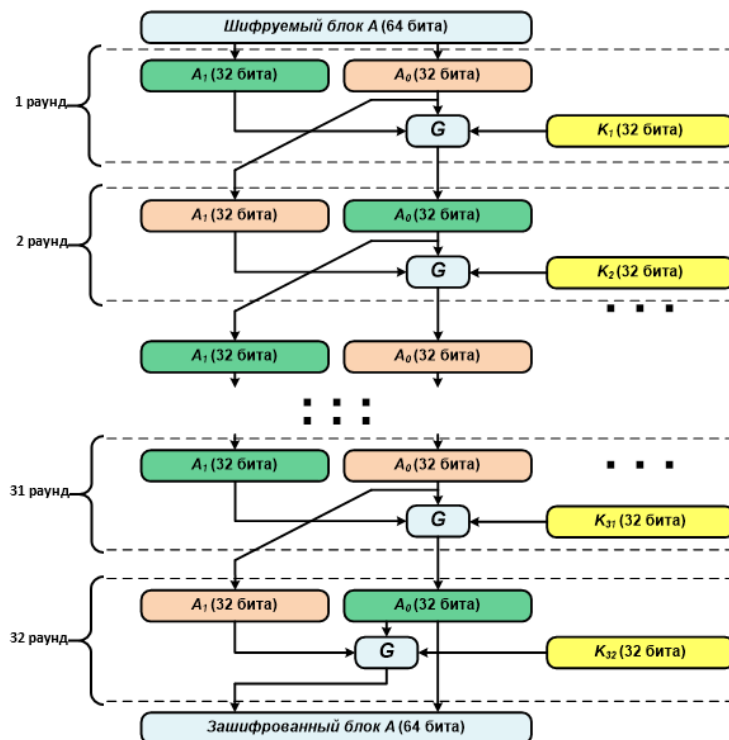


Рисунок 9 – Алгоритм шифрования Магма на основе сети Фейстеля
(источник: Яндекс картинки)

6. RSA (Rivest-Shamir-Adleman) см. рисунок 10

- Описание: Асимметричный алгоритм шифрования, использующий пару ключей: открытый для шифрования и закрытый для дешифрования. Основывается на сложности факторизации больших чисел;
- Преимущества: Обеспечивает высокую безопасность и используется для защиты данных и цифровых подписей;
- Недостатки: Медленный по сравнению с симметричными алгоритмами, требует больших вычислительных ресурсов.

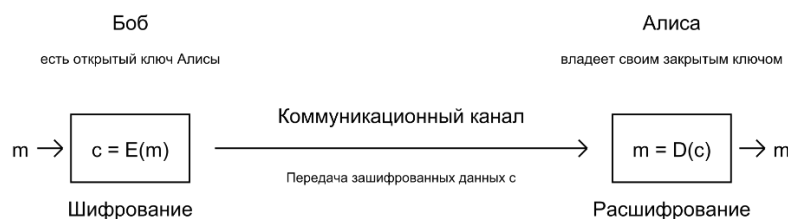


Рисунок 10 – Алгоритм шифрования RSA
(источник: Яндекс картинки)

7. Схема Эль-Гамаль, см. рисунок 11

- Описание: Асимметричный алгоритм, основанный на сложности вычисления дискретного логарифма в конечных полях. Используется для шифрования и цифровой подписи;
- Преимущества: Высокая безопасность и гибкость;
- Недостатки: Большой размер шифротекста по сравнению с RSA и более медленная скорость.

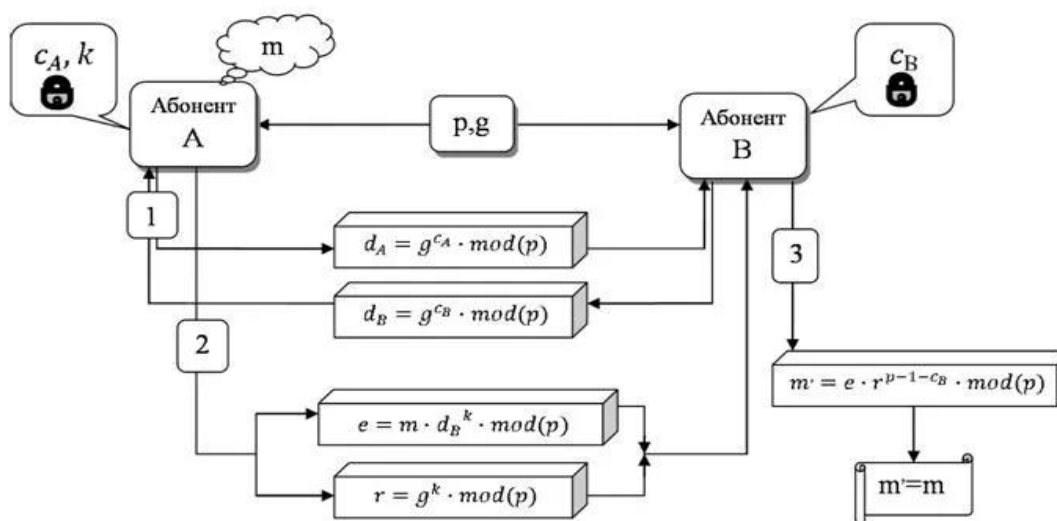
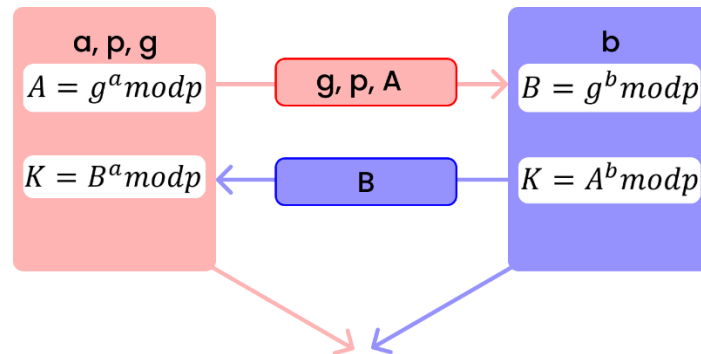


Рисунок 11 – Алгоритм шифрования Эль-Гамаля
(источник: Яндекс картинки)

8. Протокол обмена ключами Диффи-Хеллмана, см. рисунок 12.

- Описание: Позволяет двум сторонам безопасно обмениваться секретным ключом через незащищенный канал. Основывается на сложности вычисления дискретного логарифма

- Преимущества: Высокая безопасность, широко используется для создания защищенных каналов связи;
- Недостатки: Уязвим к атакам "человек посередине", если не используются дополнительные меры аутентификации.



$$K = B^a \bmod p = (g^b \bmod p)^a \bmod p = g^{ab} \bmod p$$

$$K = A^b \bmod p = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p$$
 Рисунок 12 – Протокол обмена ключами Диффи-Хеллмана

Эти алгоритмы представляют собой основу криптографии и широко применяются для защиты данных в различных системах и приложениях. Изучение этих алгоритмов важно для понимания принципов шифрования и обеспечения безопасности информации.

1.3 Обзор имеющихся образовательных ресурсов

Смартфоны в наше время являются одним из важнейших инструментов обучения, их преимущество доступ к обучению в любое время. Было скачано в количестве 6 приложений в Play Market: «Зашифровать Расшифровать» [13], «Crypto - Encryption Tools» [14], «Enigma - Криптография и хэш» [15], «Text Converter Encoder Decoder» [16], «Шифры! Морзе, Вижнер, Цезарь» [17] и «Encrypt Decrypt Tools» [18] и произведён анализ вышеперечисленных приложений. Приложений множество, но они повторяются. В ходе анализа были рассмотрены 3 приложений, а именно первые 3 соответственно, последние три практически идентичны с первыми тремя.

В приложении «Зашифровать Расшифровать» есть практическая часть, теоретическая часть отсутствует. В этом приложений пользователь вводит входные данные и получает только результат шифрования, см. рисунок 13.

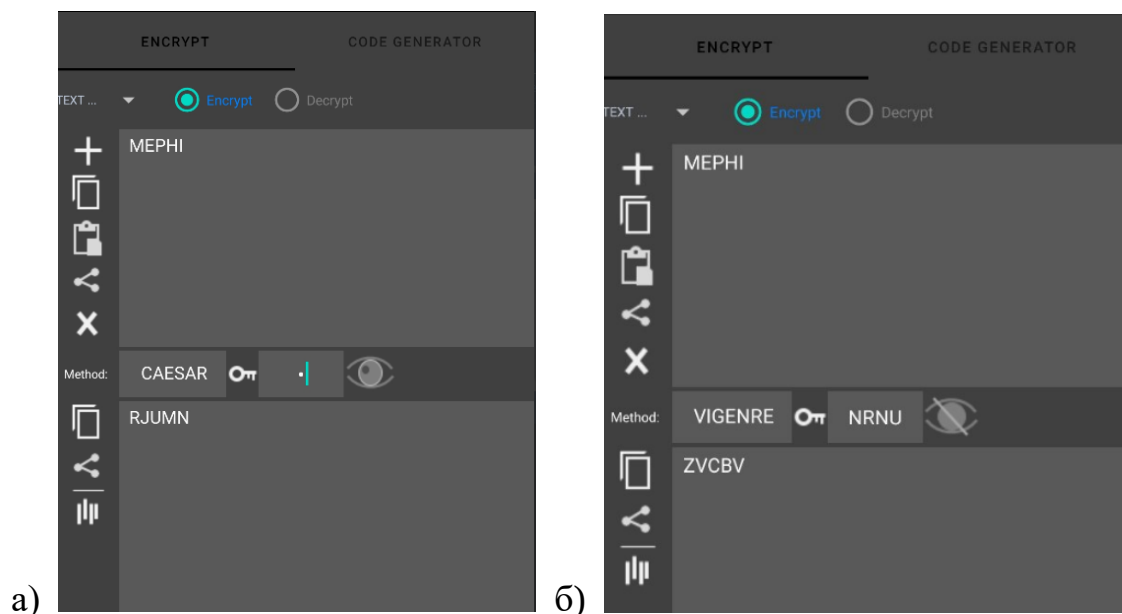


Рисунок 13 – Окно с алгоритмами шифрования. Шифр Цезаря (а), Шифр Виженера (б)
(источник: снимок экрана приложения «Зашифровать Расшифровать»)

В приложении «Crypto - Encryption Tools» отсутствует теоретическая часть. В этом приложений пользователь вводит входные данные и получает только результат шифрования, см. рисунок 14.

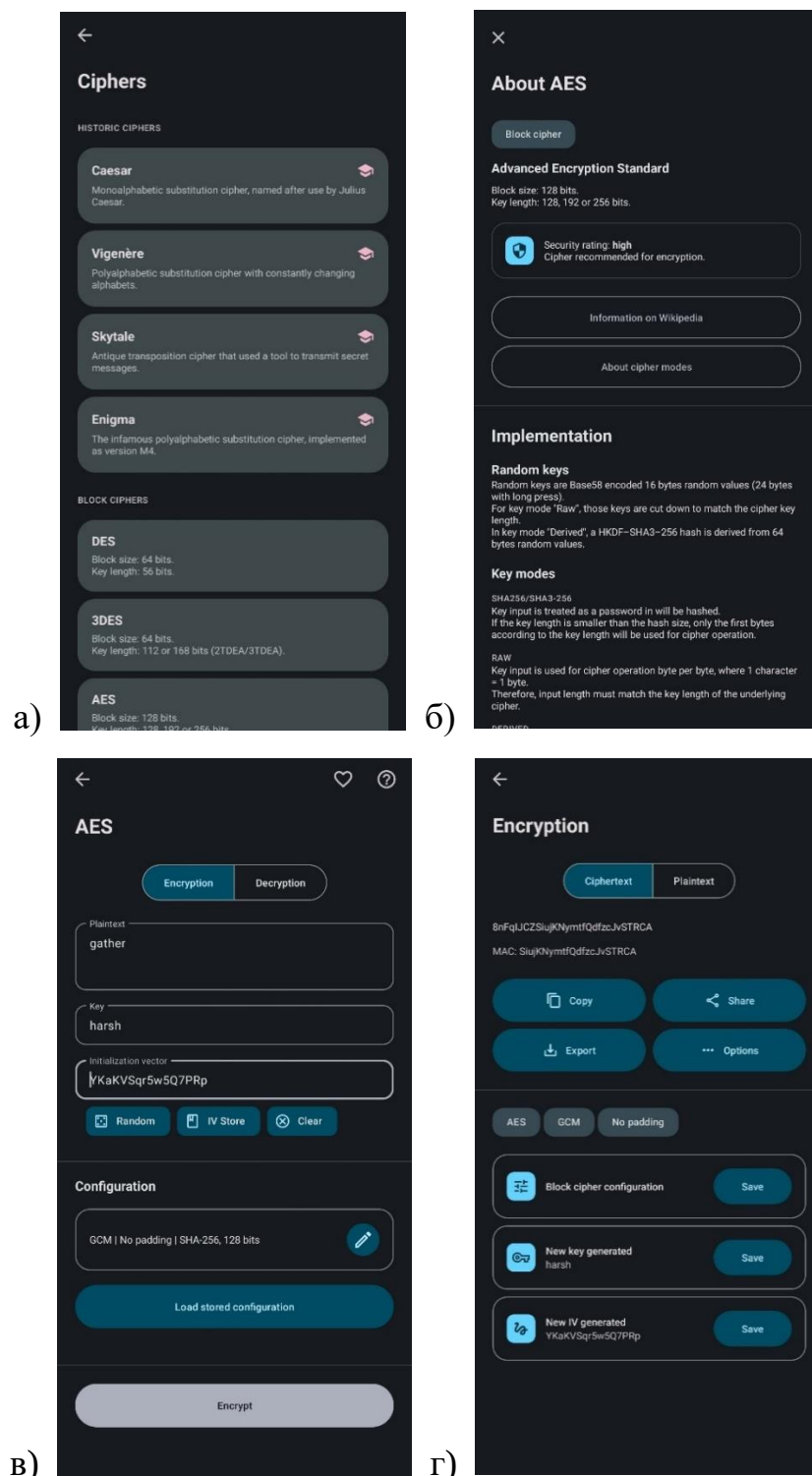
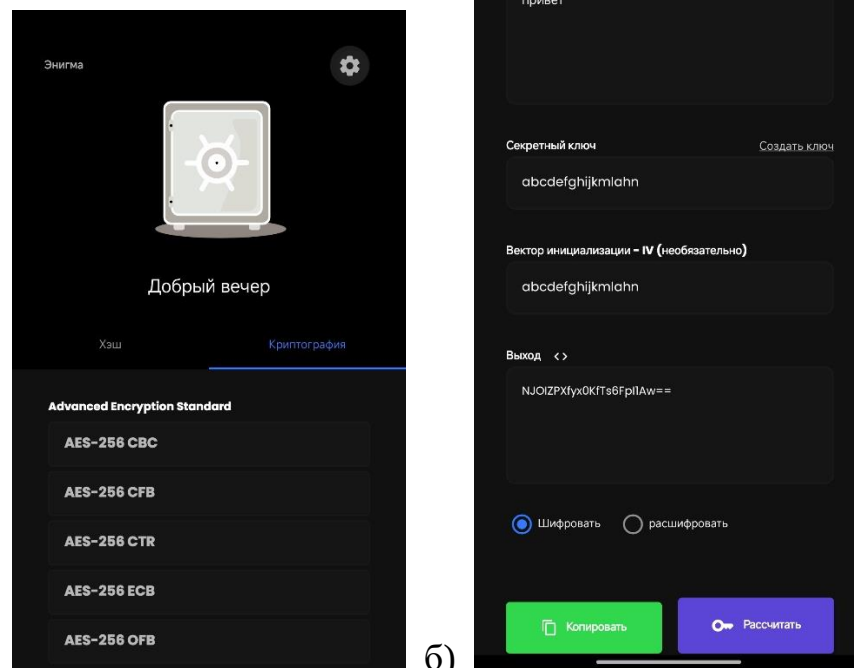


Рисунок 14 – Приложение «Crypto - Encryption Tools». Главный экран (а), об алгоритме AES (б), экран шифрования данных AES (в) и результат шифрования AES (г) (источник: снимок экрана приложения «Crypto - Encryption Tools»)

В приложении «Enigma - Криптография и хэш» отсутствует теоретическая часть. В этом приложений пользователь вводит входные данные и получает только результат шифрования, см. рисунок 15.



а) б)
Рисунок 15 – Приложение «Enigma - Криптография и хэш». Главный экран (а) и экран шифрования AES (б)
(источник: снимок экрана приложения «Enigma - Криптография и хэш»)

1.4 Целевая аудитория разрабатываемого приложения

Студенты и преподаватели информатики/кибербезопасности: Приложение может служить отличным дополнением к учебным курсам по криптографии, предоставляя интерактивный и практический опыт.

Разработчики программного обеспечения: Знание криптографии становится все более важным для разработчиков, особенно для тех, кто работает с чувствительными данными.

Любители криптографии и безопасности: Приложение предоставит им возможность углубить свои знания в интерактивной форме. Пользователи современных технологии, заинтересованные в защите своей информации:

Приложение, поможет им понять принципы шифрования и научиться создавать более надежные пароли.

1.5 Сравнение проанализированных мобильных приложений

Сравнение выбранных мобильных приложений для Android смартфонов проводится осознанно и без предвзятости, поскольку целью является выявление как преимуществ, так и недостатков каждого приложения. Это необходимо для создания, наиболее уникального и превосходящего результате разработки, который удовлетворит потребности пользователей. Выбранные приложения отличаются друг от друга и имеют свои особенности.

Сравнение будет проводиться по следующим параметрам:

- Количество алгоритмов шифрования: Число алгоритмов шифрования;
- Теоретическое описание алгоритма шифрования: Оценка наличия и качества теоретической информации о криптографических алгоритмах шифрования, представленной в приложениях;
- Приложение для обучения: разработан ли в целях обучения;
- Метод обучения: какие методы обучения присутствуют в приложениях;
- Стоимость: Цена приложения.

1. Мобильное приложения по криптографии для Android смартфонов «Зашифровать Расшифровать» [13]

- Количество алгоритмов шифрования: 5;
- Теоретическое описание алгоритма шифрования: Отсутствует;
- Приложение для обучения: Не является;
- Метод обучения: Отсутствует;
- Стоимость: Бесплатное.

2. Мобильное приложения по криптографии для Android смартфонов «Crypto - Encryption Tools» [14]

- Количество алгоритмов шифрования: 16;
- Теоретическое описание алгоритма шифрования: Частично присутствует;
- Приложение для обучения: Не является;
- Метод обучения: Отсутствует;
- Стоимость: Условно-Бесплатное.

3. Мобильное приложения по криптографии для Android смартфонов «Enigma - Криптография и хэш» [15]

- Количество алгоритмов шифрования: 5;
- Теоретическое описание алгоритма шифрования: Отсутствует;
- Приложение для обучения: Не является;
- Метод обучения: Отсутствует;
- Стоимость: Бесплатное.

В таблице 1, показано сравнения по вышеперечисленным критериям для выбранных мобильных приложений по криптографии.

Таблица 1 – Сравнение проанализированных приложений

Приложения Параметры	Зашифровать Расшифровать	Crypto - Encryption Tools	Enigma - Криптогра- фия и хэш
Количество алгоритмов шифрования	5	16	5
Теоретическое описание алгоритма шифрования	Отсутствует	Частично присут- ствует	Отсутствует
Приложение для обучения	Не является	Не является	Не является
Метод обучения	Отсутствует	Отсутствует	Отсутствует
Стоимость	Бесплатное	Условно-Бесплатное	Бесплатное

Исходя из таблицы, разрабатываемое приложение должно иметь следующие характеристики:

- Удобный интерфейс: Интерфейс должен быть понятным и удобным для пользователей, как у всех трех приложений;
- Теоретическое содержание: Приложение должно включать теоретическое содержание о криптографических алгоритмах, подобно приложению «Зашифровать Расшифровать», но без доступа Интернета;
- Дизайн: Дизайн должен быть похожим на приложение «Crypto - Encryption Tools», так как его дизайн оценен как проработанный отдельно;
- Интерактивный метод обучения: Отличительной чертой должен стать интерактивный метод обучения, который отсутствует во всех трех приложениях из таблицы/

Итак, разрабатываемое приложение должно сочетать удобный интерфейс и теоретическое содержание, иметь дизайн, схожий с приложением «Crypto - Encryption Tools», и предлагать интерактивный метод обучения, что делает его востребованным и уникальным среди мобильных приложений для Android смартфонов для обучения алгоритмам криптографического шифрования.

2 Моделирование структуры приложения

2.1 Описание выбранных инструментов и технологий для разработки

Kotlin - это относительно молодой язык от российской компании JetBrains. Появился он в 2011 году. На конференции Google I/O 2017 команда разработчиков Android сообщила, что Kotlin получил официальную поддержку для разработки Android - приложений.

Kotlin представляет собой современный, статически типизированный язык программирования, который быстро развивается благодаря усилиям компании JetBrains. Он подходит для создания разнообразных приложений, включая мобильные (Android и iOS), кроссплатформенные, веб - приложения (как серверные для бэкенда, так и клиентские для фронтенда), десктопные приложения и разработки в области Data Science.

Таким образом, спектр платформ, на которых можно использовать Kotlin, чрезвычайно широк: Windows, Linux, Mac OS, iOS, Android.

Наиболее популярной областью применения Kotlin является разработка для операционной системы Android. Его популярность возросла настолько, что на конференции Google I/O 2017 компания Google объявила Kotlin одним из официальных языков разработки для Android наряду с Java и C++, и начиная с версии Android Studio 3.0, инструменты для работы с Kotlin были интегрированы в Android Studio по умолчанию.

Официальный сайт языка, <https://kotlinlang.org/>, предоставляет самую актуальную и подробную информацию о Kotlin.

Первая версия Kotlin была выпущена 15 февраля 2016 года, хотя разработка началась еще в 2010 году. На данный момент актуальной является версия 1.9, выпущенная в июле 2023 года.

Название языка происходит от острова Kotlin, расположенного недалеко от Санкт-Петербурга, где и началась его разработка.

Kotlin вобрал в себя черты многих языков, таких как Java, Scala, Groovy, C#, JavaScript, Swift, и поддерживает как объектно-ориентированный, так и функциональный стили программирования. Он обладает ясным и понятным синтаксисом, что делает его легким для изучения [19].

Kotlin - это не просто еще один язык программирования, а целая экосистема, см. рисунок 16.

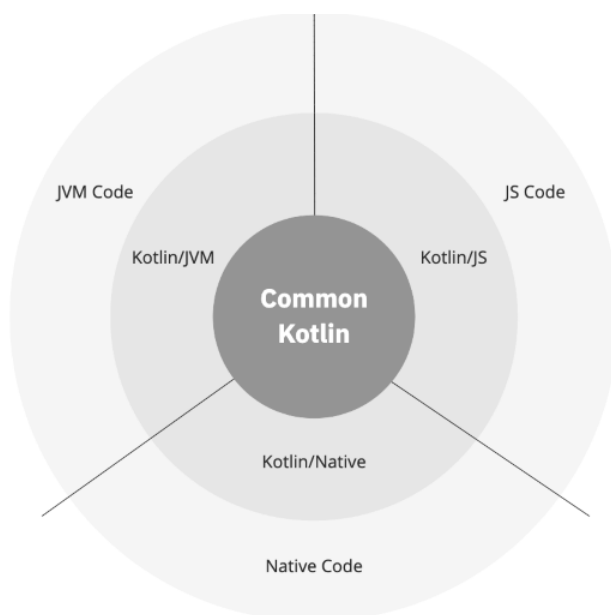


Рисунок 16 – Экосистема языка программирования Kotlin
(источник: metanit.com)

Kotlin - это многофункциональный язык программирования, который поддерживает несколько целевых платформ. Это позволяет разработчикам использовать один и тот же код на разных платформах, таких как JVM, JavaScript и Native. Рассмотрим каждую из этих платформ подробнее.

Common Kotlin - это часть кода, которая может быть использована на всех платформах, поддерживаемых Kotlin. Этот код содержит общую бизнес-логику, которая не зависит от конкретной платформы.

- Преимущества: Позволяет писать код, который можно повторно использовать на различных платформах (например, общая логика для мобильных и веб-приложений);
- Примеры использования: Модели данных, логика валидации, общие функции и утилиты.

Kotlin/JVM - это вариант Kotlin, который компилируется в байт-код, исполняемый на Java Virtual Machine (JVM). Это позволяет Kotlin - коду работать в любом месте, где работает JVM, включая Android.

- Использование: Разработка серверных приложений, веб - приложений, библиотек и приложений для Android;
- Совместимость: Полная совместимость с существующим Java - кодом и библиотеками;
- Преимущества: Широкая экосистема Java, возможность использовать обширные библиотеки и фреймворки Java.

Kotlin/JS - это вариант Kotlin, который компилируется в JavaScript. Это позволяет использовать Kotlin для разработки веб-приложений и работать в браузере.

JS Code в контексте Kotlin/JS относится к JavaScript-коду, который получается в результате компиляции кода на языке Kotlin в JavaScript. Это позволяет использовать язык Kotlin для разработки клиентских веб - приложений, которые выполняются в браузере. Использование: Разработка клиентских веб-приложений, интерактивных веб-интерфейсов.

- Совместимость: Возможность использовать JavaScript - библиотеки и взаимодействовать с существующим JavaScript – кодом;
- Преимущества: Совместное использование логики между фронтендом и бекендом (если бекенд написан на Kotlin/JVM).

Kotlin/Native - это вариант Kotlin, который компилируется в машинный код и может выполняться без необходимости в виртуальной машине. Это

позволяет создавать приложения, которые работают напрямую на операционной системе.

Native Code - это технология, позволяющая компилировать код на языке Kotlin в нативный машинный код, который может выполняться на различных платформах без необходимости в виртуальной машине (JVM). Это позволяет создавать приложения, которые могут работать на iOS, macOS, Windows, Linux и других платформах.

- Использование: Разработка десктопных приложений, приложений для iOS, системного программного обеспечения;
- Совместимость: Возможность интеграции с библиотеками, написанными на других языках (например, C, Objective - C);
- Преимущества: Высокая производительность и возможность создания приложений для платформ, где виртуальная машина не подходит (например, iOS).

Android Studio - интегрированная среда разработки (IDE) для работы с платформой Android, анонсированная 16 мая 2013 года на конференции Google I/O.

Данная IDE находилась в свободном доступе начиная с версии 0.1, опубликованной в мае 2013, а затем перешла в стадию бета - тестирования, начиная с версии 0.8, которая была выпущена в июне 2014 года. Первая стабильная версия 1.0 была выпущена в декабре 2014 года, тогда же прекратилась поддержка плагина Android Development Tools (ADT) для Eclipse.

Android Studio, основанная на программном обеспечении IntelliJ IDEA от компании JetBrains - официальное средство разработки Android приложений. Данная среда разработки доступна для Windows, macOS и GNU/Linux. 17 мая 2017, на ежегодной конференции Google I/O, Google анонсировал поддержку языка Kotlin, используемого в Android Studio, как официального языка программирования для платформы Android в дополнение к Java и C++. Основные возможности и особенности Android Studio [20].

1. Редактор кода:

- Авто дополнение: Предоставляет интеллектуальное авто дополнение кода для Java, Kotlin и XML, что ускоряет процесс написания кода и снижает количество ошибок;
- Рефакторинг: Инструменты рефакторинга помогают безопасно изменять структуру кода, сохраняя его работоспособность;
- Навигация по коду: Легко перемещаться по коду благодаря навигационным инструментам, таким как переход к определению, просмотр использований и другие.

2. Редактор пользовательского интерфейса:

- Визуальный редактор макетов: Поддержка drag-and-drop для создания и редактирования макетов интерфейса;
- Редактор ConstraintLayout: Удобный инструмент для создания сложных и гибких макетов, адаптирующихся под разные размеры экранов.

3. Инструменты сборки:

- Gradle: Использует систему сборки Gradle, что позволяет автоматизировать процесс сборки, управления зависимостями и конфигураций;
- Возможность создания различных конфигураций сборок: Например, debug и release конфигурации, которые могут включать различные настройки и ресурсы.

4. Инструменты отладки и тестирования:

- Android Emulator: Встроенный эмулятор для тестирования приложений на различных устройствах и версиях Android;
- Профилирование производительности: Инструменты для анализа производительности приложения, включая мониторинг использования CPU, памяти, сети и графики;

- Инструменты для автоматизированного тестирования: Поддержка JUnit, Espresso, UI Automator и других фреймворков для написания и выполнения тестов.
5. Поддержка Kotlin:
- Интеграция с Kotlin: Полная поддержка языка Kotlin, включая авто дополнение, рефакторинг и анализ кода;
 - Миграция кода: Инструменты для легкой миграции существующего кода с Java на Kotlin.
6. Поддержка версионного контроля:
- Git, SVN и другие: Интеграция с системами контроля версий, что позволяет управлять репозиториями прямо из IDE;
 - Встроенные инструменты для управления версиями: Возможность выполнять коммиты, слияния и другие операции с версионными контролями.
7. Поддержка плагинов и расширений:
- Плагины: Возможность установки дополнительных плагинов для расширения функциональности IDE;
 - Интеграция с сервисами: Поддержка различных сервисов, таких как Firebase, для добавления аналитики, базы данных и других функций в приложение.

2.2 Процесс моделирования структуры приложения

Названием приложения выбрано «CryptoLearn». На главном экране будет представлен список типов и алгоритмов шифрования. Одним коротким нажатием на название алгоритма или типа шифрования можно будет открыть теоретическое описание (соответствующее activity), а при длительном нажатии на название алгоритма перейти к экрану с практическим интерактивным шифрованием (соответствующее activity).

Список состоит из следующих алгоритмов и типов шифрования:

- Классические шифры: Шифр Цезаря и Лозунговый шифр;
- Симметричные шифры: AES, DES и Магма;
- Асимметричные шифры: RSA, Схема Эль-Гамала и Протокол Диффи-Хеллмана;
- Генератор паролей.

Основные функции приложения включают:

Теоретическое описание: Подробное описание принципов работы каждого алгоритма и краткое описание о типах шифрования.

Практическое интерактивное шифрование: Данные которые пользователь ввёл при шифровании и дешифровании в выводе будет отображена пошагово зашифрованным или дешифрованным, т.е. можно будет изучить каждый этап вычисления. Алгоритм покажет шаг за шагом как были зашифрованы или дешифрованы введённые пользователем данные.

Структура приложения, см. рисунок 17.

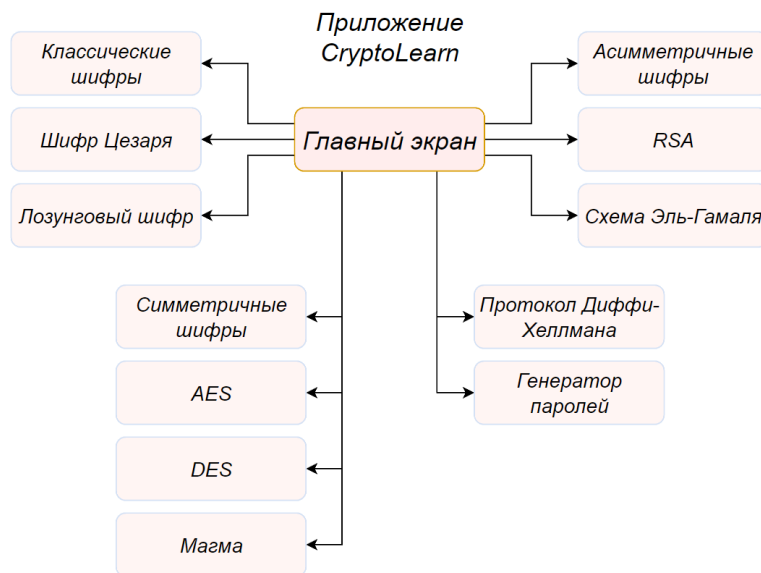


Рисунок 17 – Структура мобильного приложения «CryptoLearn»

3 Процесса разработки мобильного приложения и инструкция пользования приложением

3.2 Установка и настройка необходимых программных средств

Первоначально необходимо установить следующие программные обеспечения: JDK, IntelliJ IDEA и Android Studio. Операционная система для установки вышеперечисленных программных обеспечений Microsoft Windows 11 Pro. Соответственно, все устанавливаемые программные обеспечения должны быть совместимы с этой версией операционной системы.

JDK - Java Development Kit (сокр. JDK) бесплатно распространяемый корпорацией Oracle Corporation (ранее Sun Microsystems) комплект разработчика приложений на языке Java, включающий в себя компилятор Java (javac), стандартные библиотеки классов Java, примеры, документацию, различные утилиты и исполнительную систему Java (JRE). В состав JDK не входит интегрированная среда разработки на Java, поэтому разработчик, использующий только JDK, вынужден использовать внешний текстовый редактор и компилировать свои программы, используя утилиты командной строки.

Все современные интегрированные среды разработки приложений на Java, такие, как JDeveloper, NetBeans IDE, Sun Java Studio Creator, IntelliJ IDEA, Borland JBuilder, Eclipse, опираются на сервисы, предоставляемые JDK. Большинство из них для компиляции Java-программ используют компилятор из комплекта JDK. Поэтому эти среды разработки либо включают в комплект поставки одну из версий JDK, либо требуют для своей работы предварительной инсталляции JDK на машине разработчика [21].

Для установки JDK (Java Development Kit) на Windows необходимо выполнить следующие шаги:

1. Скачать JDK совместимой с операционной системой
 - Перейти на официальный сайт Oracle JDK <https://www.oracle.com/>, открыть раздел JDK;

- Выбрать последнюю версию JDK и нажать на ссылку скачивания установочного файла для Windows (обычно расширение файла `.exe`) см. рисунок 18.

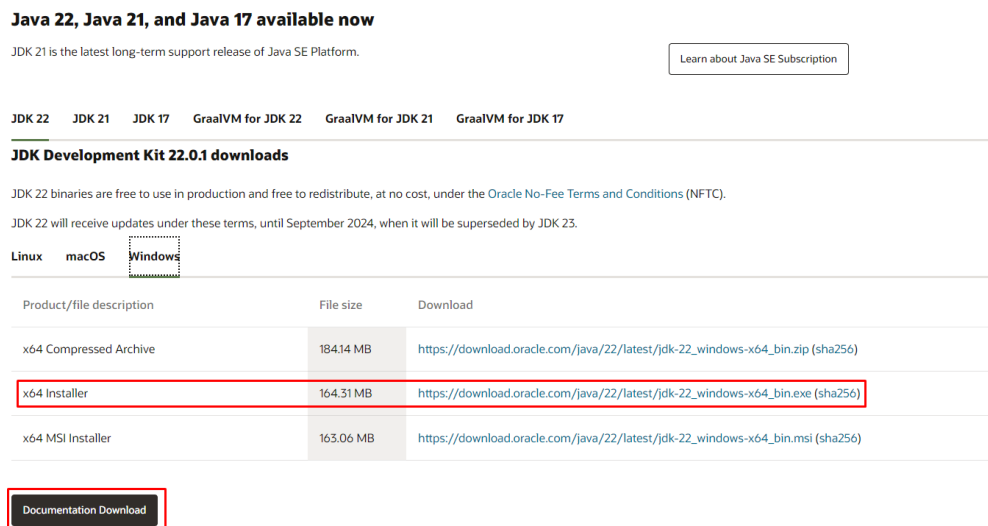


Рисунок 18 – Официальный сайт Oracle JDK, скачивание установочного файла для Windows (источник: oracle.com)

При необходимости можно скачать и изучить документацию по JDK.

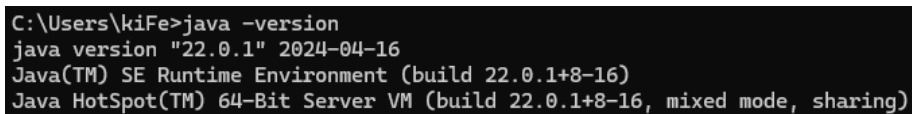
2. Установка JDK.

- Запустить скачанный установочный файл;
- Изучить и принять лицензионное соглашение;
- Указать путь установки или оставить по умолчанию;
- Дождаться завершения установки и нажать кнопку «Close» см. рисунок 19.



Рисунок 19 – Успешное завершение установки JDK последней версии

Проверить установку можно проверить в командной строке Windows командой `java -version`, см. рисунок 20.



```
C:\Users\kiFe>java -version
java version "22.0.1" 2024-04-16
Java(TM) SE Runtime Environment (build 22.0.1+8-16)
Java HotSpot(TM) 64-Bit Server VM (build 22.0.1+8-16, mixed mode, sharing)
```

Рисунок 20 – Проверка на наличие установки JDK

В данном проекте в качестве языка программирования будет использован язык программирования Kotlin. Kotlin может использовать все возможности и библиотеки JDK.

Kotlin - это язык программирования, созданный в компании JetBrains. Его разработали в 2011 году на замену Java, который в компании считали чересчур многословным. Новый язык получился на 40% компактнее предшественника, что помогло ускорить работу над основным продуктом JetBrains - средой разработки IntelliJ IDEA. При этом Kotlin полностью совместим с Java, потому что запускается на его виртуальной машине (JVM).

Если же давать формальное определение, Kotlin - это объектно-ориентированный язык со статической типизацией. Что это значит?

Объектно-ориентированными называют языки, в которых все операции происходят с объектами - блоками кода, куда можно «складывать» несколько значений. Объектом может быть любая сущность с определённым набором характеристик. Например, персонаж в онлайн-игре или стикер со Спанч Бом в Telegram. Все объекты проектируются по специальным шаблонам, которые называются классами.

Статическая типизация означает, что типы переменных задаются разработчиком до выполнения программы. Если вы объявили целочисленную переменную, текст в неё поместить уже не получится - компилятор сразу укажет на ошибку. Это замедляет разработку на начальном этапе, но добавляет надёжности [22].

IntelliJ IDEA. Ведущая IDE для разработки на Java и Kotlin. IntelliJ IDEA обеспечивает уникальный уровень поддержки Java и Kotlin, в том числе с самого начала поддерживает новейшие возможности языка, позволяя вам опередить своих конкурентов. IntelliJ IDEA знает все о вашем коде и использует эти знания для молниеносной навигации и умных подсказок, предоставляя подходящие варианты авто дополнения в любом контексте. С IntelliJ IDEA все ключевые инструменты уже под рукой, а значит, не нужно тратить время на поиск и установку плагинов для работы с определенным языком или технологией. Решение учитывает потребности крупных компаний на всех этапах работы: от проектирования и разработки до тестирования и развертывания. 78% Java/Kotlin-разработчиков выбирают IntelliJ IDEA [23].

Для установки IntelliJ IDEA на Windows необходимо выполнить следующие шаги:

1. Скачать IntelliJ IDEA совместимой с операционной системой
 - Перейти на официальный сайт JetBrains и выбрать среди продуктов IntelliJ IDEA <https://www.jetbrains.com/ru-ru/>;
 - Для разработки приложения достаточно бесплатной версии IntelliJ IDEA Community Edition;
 - Выбрать последнюю версию IntelliJ IDEA Community Edition и нажать на ссылку скачивания установочного файла для Windows (обычно расширение файла `.exe`), см. рисунок 21.

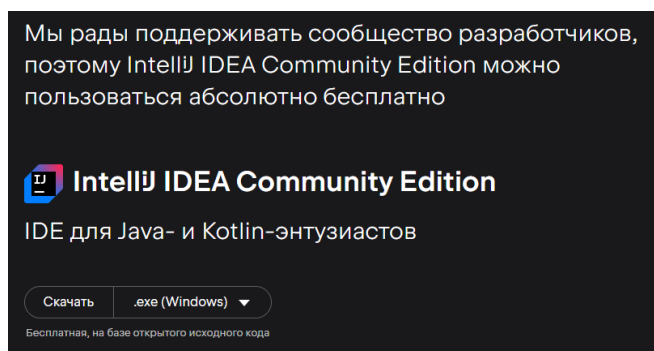


Рисунок 21 – Официальный сайт JetBrains, скачивание установочного файла для Windows (источник: jetbrains.com)

2. Установка IntelliJ IDEA Community Edition.

- Запустить скачанный установочный файл;
- Изучить и принять лицензионное соглашение;
- Указать путь установки или оставить по умолчанию;
- По желанию создать иконку для быстрого запуска;
- Дождаться завершения установки и нажать кнопку «Finish», см. рисунок 22.



Рисунок 22 – Успешное завершение установки IntelliJ IDEA Community Edition последней версии

Android Studio - это интегрированная среда разработки (integrated development environment) от Google для создания Android - приложений. В ней можно писать код, проектировать графический интерфейс, проводить отладку и сборку приложений для публикации в магазинах.

Код в Android Studio пишут на Java или Kotlin - языке программирования, который в Google назвали официальным для разработки под операционную систему Android.

В эмуляторе операционной системы можно запустить готовые пресеты виртуального железа, которые работают аналогично реально существующим

устройствам, или создать собственные конфигурации с определённой версией операционной системы. Это позволяет протестировать приложение на десятках разных устройств.

Когда код будет написан и протестирован, Android Studio приведёт его к формату, подходящему для публикации в Google Play, AppGallery, RuStore и других магазинах приложений для Android [24].

Для установки Android Studio на Windows необходимо выполнить следующие шаги:

1. Скачать Android Studio совместимой с операционной системой
 - Перейти на официальный сайт Android Studio developer.android.com/studio;
 - Нажать на ссылку скачивания установочного файла для Windows (обычно расширение файла `.exe`), см. рисунок 23.

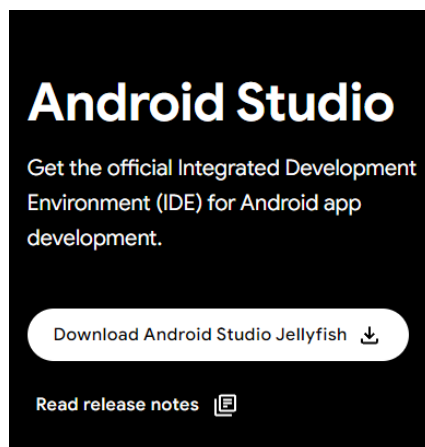


Рисунок 23 – Официальный сайт Android Studio, скачивание установочного файла для Windows
(источник: developer.android.com)

2. Установка Android Studio.

- Запустить скачанный установочный файл;
- Изучить и принять лицензионное соглашение;
- Указать путь установки или оставить по умолчанию;
- По желанию создать иконку для быстрого запуска;

- Дождаться завершения установки и нажать кнопку «Finish», см. рисунок 24.

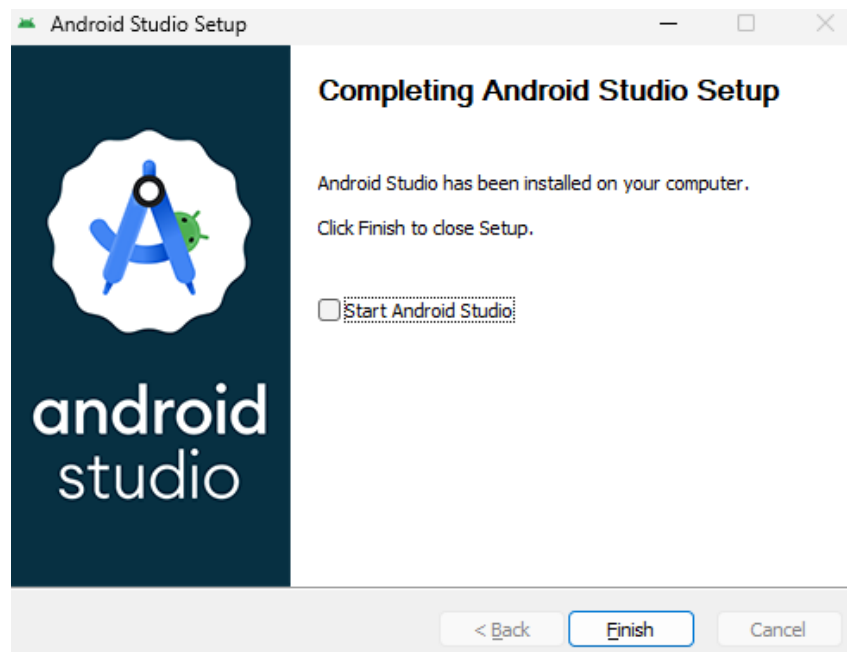


Рисунок 24 – Успешное завершение установки Android Studio последней версии

Затем, необходимо установить средство для разработки Android SDK. Было установлено следующие SDK платформы: Android 8, 8.1, 9, 10, 11, 12, 12L, 13, 14. Чтобы данное приложение поддерживало большое количество смартфонов.

3.3 Алгоритмы шифрования дешифрования. Написание и тестирование алгоритмов

После установки всех необходимых приложений можно приступить к разработке мобильного приложения для Android смартфонов.

JDK был установлен для поддержки языка программирования Kotlin. IntelliJ IDEA был установлен для написания кода алгоритмов криптографического шифрования, при этом в IntelliJ IDEA можно легко проверить работоспособность кода, проверить на наличие различных багов и исправления различных ошибок. Android Studio был установлен для разработки приложения. Интерфейс и функционал разрабатывается в данном приложений. В конеч-

ном итоге после написания кодов шифрования, разработки функционала и интерфейса, можно произвести слияние для получения конечного продукта.

Алгоритмы шифрования «Шифр Цезаря», «Лозунговый шифр», «AES», «DES», «Магма», «RSA» «Схема Эль-Гамала» и протокол обмена ключами «Протокол Диффи-Хеллмана», а также «Генератор паролей» были написаны на языке программирования Kotlin и протестированы в IntelliJ IDEA, см. рисунок 26.

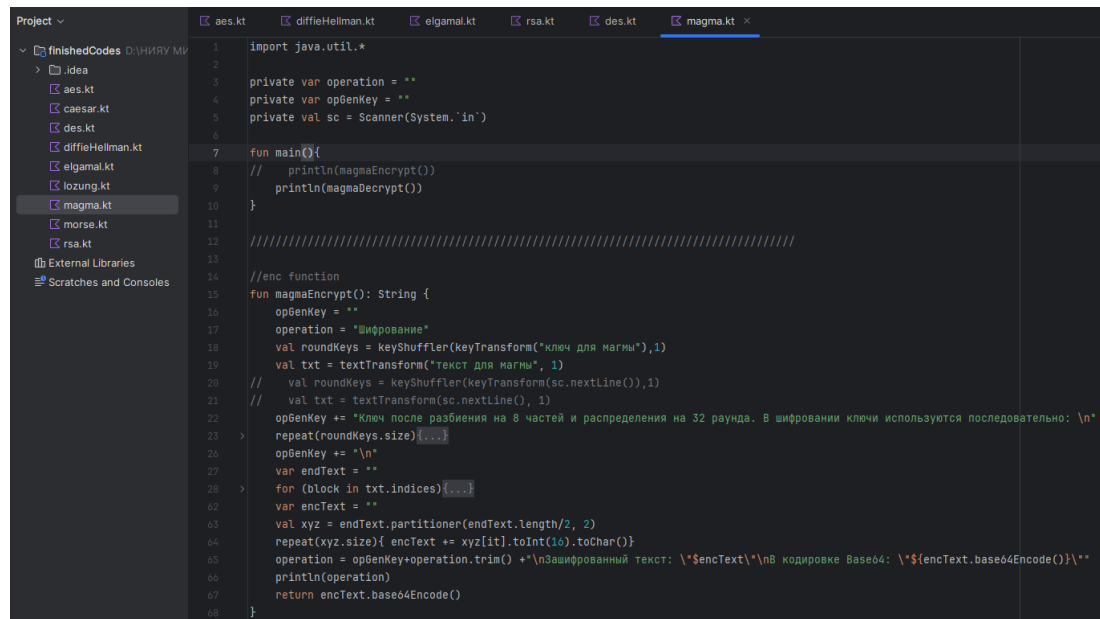
Исходные коды приведены в приложении ВКР:

- Приложение А. Алгоритм шифрования «Шифр Цезаря»,
- Приложение Б. Алгоритм шифрования «Лозунговый шифр»,
- Приложение В. Алгоритм шифрования «AES»,
- Приложение Г. Алгоритм шифрования «DES»,
- Приложение Д. Алгоритм шифрования «Магма»,
- Приложение Ж. Алгоритм шифрования «RSA»,
- Приложение И. Алгоритм шифрования «Схема Эль-Гамала»,
- Приложение К. Алгоритм обмена ключами «Протокол обмена ключами Диффи-Хеллмана».

Для написания алгоритмов были изучены следующие интернет ресурсы [25], [26], [27], [28], [29], [30], [31], [32], [33] и книги [1], [2], [3], [4], [5], [6].

Написанные алгоритмы криптографического шифрования на языке программирования Kotlin в среде разработки IDE IntelliJ IDEA, см. листинги 1, 2 и 3.

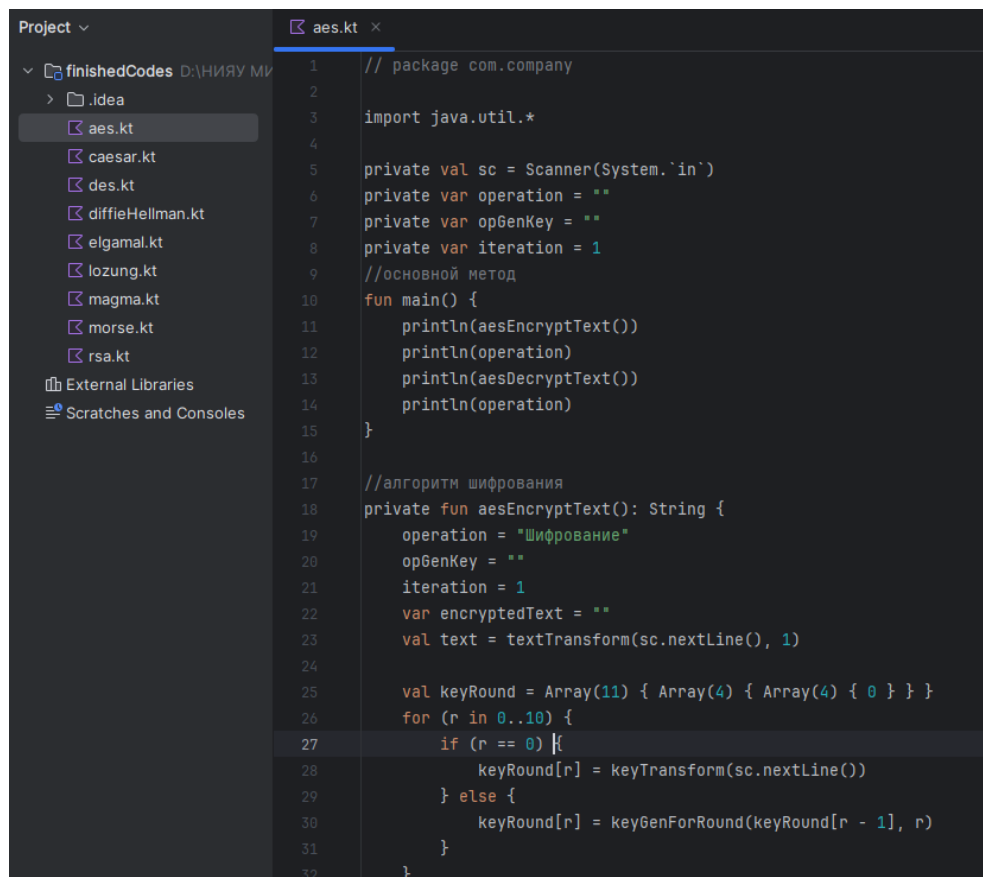
Листинг 1 – Симметричное шифрования Магма, реализованный в IDE IntelliJ IDEA



```
Project ▾
  ▾ finishedCodes D:\ИИЯУ МВ
    > .idea
      aes.kt
      caesar.kt
      des.kt
      diffieHellman.kt
      elgamal.kt
      lozung.kt
      magma.kt
      morse.kt
      rsa.kt
  External Libraries
  Scratches and Consoles

1 import java.util.*
2
3 private var operation = ""
4 private var opGenKey = ""
5 private val sc = Scanner(System.`in`)
6
7 fun main() {
8     // println(magmaEncrypt())
9     println(magmaDecrypt())
10 }
11
12 ///////////////////////////////////////////////////////////////////
13
14 //enc function
15 fun magmaEncrypt(): String {
16     opGenKey = ""
17     operation = "Шифрование"
18     val roundKeys = keyShuffler(keyTransform("ключ для магмы"), 1)
19     val txt = textTransform("текст для магмы", 1)
20     val roundKeys = keyShuffler(keyTransform(sc.nextLine(), 1))
21     // val txt = textTransform(sc.nextLine(), 1)
22     opGenKey += "Ключ после разбиения на 8 частей и распределения на 32 раунда. В шифровании ключи используются последовательно: \n"
23     repeat(roundKeys.size) { ... }
24     opGenKey += "\n"
25     var endText = ""
26     for (block in txt.indices) { ... }
27     var encText = ""
28     val xyz = endText.partitioner(endText.length/2, 2)
29     repeat(xyz.size) { encText += xyz[it].toInt(10).toChar() }
30     operation = opGenKey+operation.trim() + "\n3ашифрованный текст: \"$encText\"\n8 кодировке Base64: \"${encText.base64Encode()}\""
31     println(operation)
32     return encText.base64Encode()
33 }
```

Листинг 2 – Симметричное шифрования AES, реализованный в IDE IntelliJ IDEA



```
Project ▾
  ▾ finishedCodes D:\ИИЯУ МВ
    > .idea
      aes.kt
      caesar.kt
      des.kt
      diffieHellman.kt
      elgamal.kt
      lozung.kt
      magma.kt
      morse.kt
      rsa.kt
  External Libraries
  Scratches and Consoles

1 // package com.company
2
3 import java.util.*
4
5 private val sc = Scanner(System.`in`)
6 private var operation = ""
7 private var opGenKey = ""
8 private var iteration = 1
9 //основной метод
10 fun main() {
11     println(aesEncryptText())
12     println(operation)
13     println(aesDecryptText())
14     println(operation)
15 }
16
17 //алгоритм шифрования
18 private fun aesEncryptText(): String {
19     operation = "Шифрование"
20     opGenKey = ""
21     iteration = 1
22     var encryptedText = ""
23     val text = textTransform(sc.nextLine(), 1)
24
25     val keyRound = Array(11) { Array(4) { Array(4) { 0 } } }
26     for (r in 0..10) {
27         if (r == 0) {
28             keyRound[r] = keyTransform(sc.nextLine())
29         } else {
30             keyRound[r] = keyGenForRound(keyRound[r - 1], r)
31         }
32     }
33 }
```

Листинг 3 – Асимметричное шифрование RSA, реализованный в IDE IntelliJ IDEA

```

package com.company

import java.math.BigInteger
import java.security.KeyFactory
import java.security.KeyPair
import java.security.KeyPairGenerator
import java.security.PrivateKey
import java.security.PublicKey
import java.security.spec.PKCS8EncodedKeySpec
import java.security.spec.X509EncodedKeySpec
import java.util.Scanner

private var operation = ""
private var genKeyRSA = ""
private var e = 0
private var d = 0
private var mod = 0
private val sc = Scanner(System.`in`)

fun main() {
    println(genKeyForRSA())
    println(encryptRSA())
    println(decryptRSA())
}

//Шифрование
fun encryptRSA(): String {
    operation = ""
    val text = sc.nextLine()
    var encText = ""
    operation += "Шифрование исходного текста, с=(m^e)mod(n). Исходный текст: \n": "$text\n"
    for (i in text.indices) {
        encText += (text[i].hashCode().toBigInteger().pow(e).rem(mod.toBigInteger()).toInt().toChar())
        val resx = text[i].hashCode().toBigInteger().pow(e).toString()
        operation += if (resx.length > 30) {
            "Защ-код ${i+1}-го символа: \n${text[i]} \n ${0x2794.toChar()} ${text[i].hashCode()} \n" +
            "tm^e = ${text[i].hashCode()}^$e: ${resx.substring(0, 3)}...${resx.length-6} шмр...${resx.substring(resx.length-3)} \n" +
            "t(m^e)mod($mod): ${encText[i].hashCode()} \n" +
            "t${encText[i].hashCode()} ${0x2794.toChar()} \n${encText[i]} \n"
        } else {
            "Защ-код ${i+1}-го символа: \n${text[i]} \n ${0x2794.toChar()} ${text[i].hashCode()} \n" +
            "tm^e = ${text[i].hashCode()}^$e: ${text[i].hashCode().toBigInteger().pow(e)} \n" +
            "t(m^e)mod($mod): ${encText[i].hashCode()} \n" +
            "t${encText[i].hashCode()} ${0x2794.toChar()} \n${encText[i]} \n"
        }
    }
    operation += "Зашифрованный текст: \n${encText}\nЗашифрованный текст после кодировки Base64: \n${encText.base64Encode().trim()}"
    return operation
}
    
```

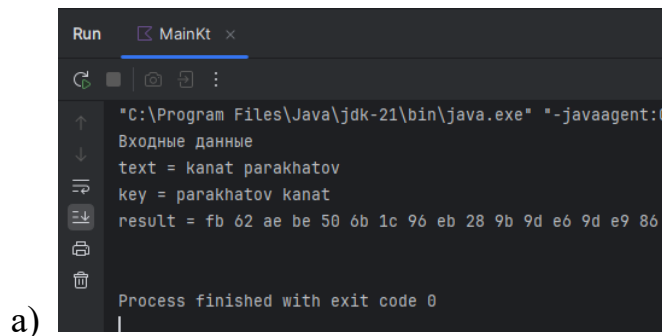
Проверка на корректность написанных алгоритмов в IDE IntelliJ IDEA:

1. Алгоритм шифрования «Шифр Цезаря», см. рисунок 25.



Рисунок 25 – Алгоритм классического шифрования «Шифр Цезаря». снимок экрана в IDE IntelliJ IDEA (а) и снимок экрана на сайте thewordfinder.com (б)
(источник: thewordfinder.com)

2. Алгоритм симметричного шифрования «AES,» см. рисунок 26.



AES – Symmetric Ciphers Online

Input type:

Input text:
(plain)

kanat parakhatov

☒ Plaintext ☐ Hex Autodetect: **ON** | OFF

Function:

Mode:

Key:
(plain)

parakhatov kanat

☒ Plaintext ☐ Hex

> Encrypt! > Decrypt!

Encrypted text:

б) 00000000 fb 62 ae be 50 6b 1c 96 eb 28 9b 9d e6 9d e9 86 | û b ° % P k . ð ë (. ð æ ð é .

Рисунок 26 – Алгоритм симметричного шифрования «AES». Снимок экрана в IDE IntelliJ IDEA (а) и снимок экрана на сайте aes.online-domain-tools.com (б)
(Источник: aes.online-domain-tools.com)

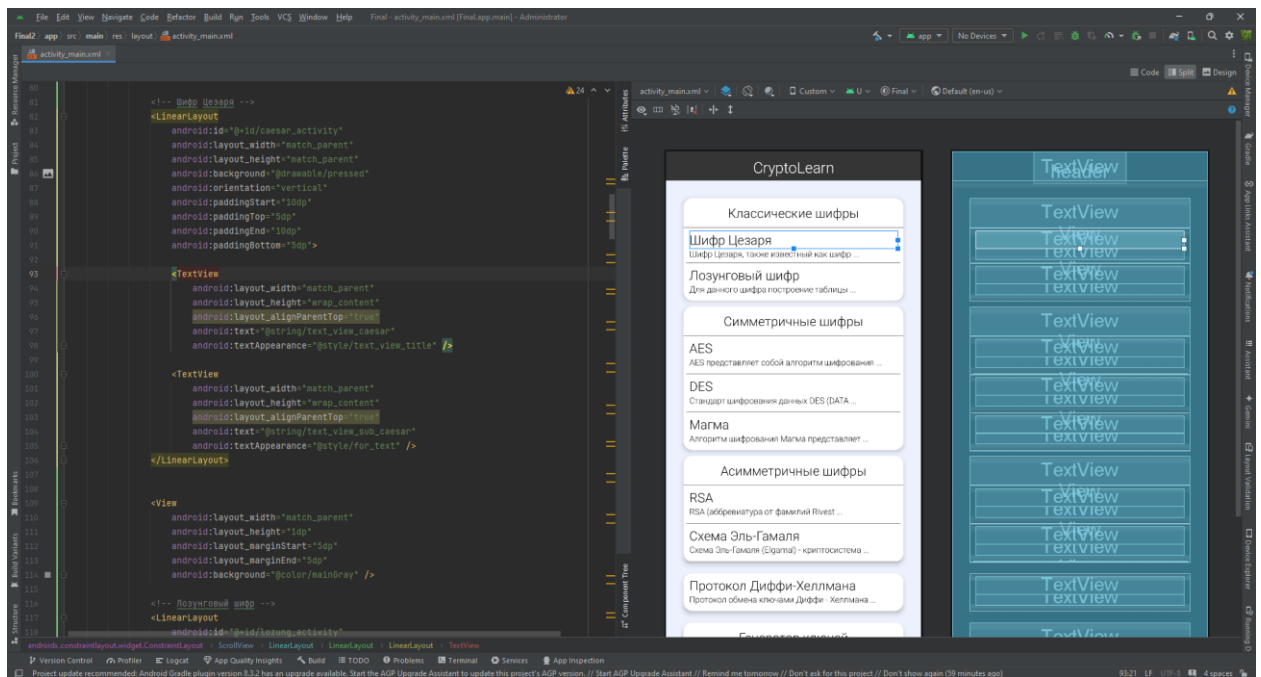
3.4 Разработка интерфейса приложения

Интерфейс приложения будет разработана в соответствии со структурой приложения. Графический интерфейс включает в себя окна, иконки, кнопки, картинки и checkbox. Текстовый интерфейс включает в себя тексты, поля ввода и вывода.

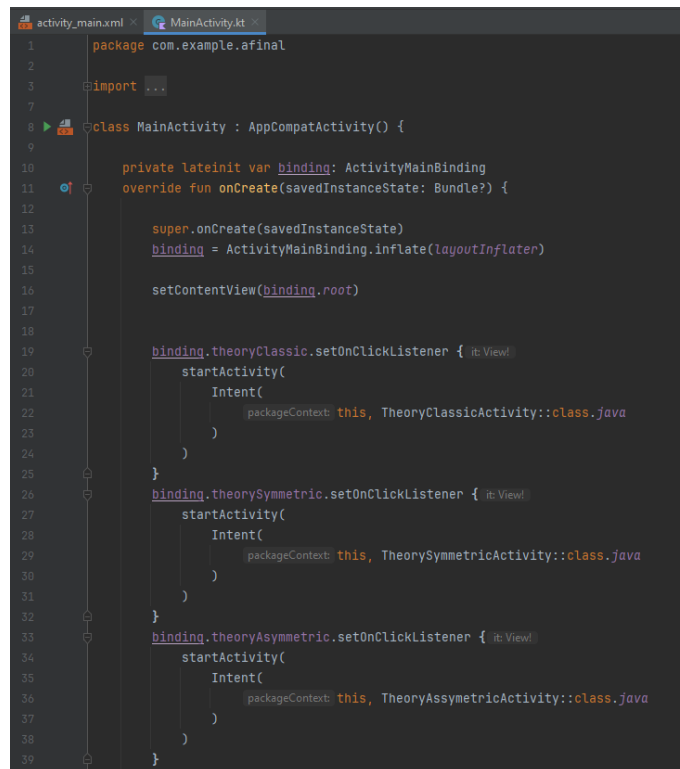
В разработке приложений для Android используются два языка: XML для разметки пользовательского интерфейса и Kotlin для написания логики приложения. В разметке для определённого элемента присваивается идентификатор, чтобы работать с ним уже в логике приложения.

Ниже представлены рисунки из IDE Android Studio, см. листинги 4, 5, 6 и 7.

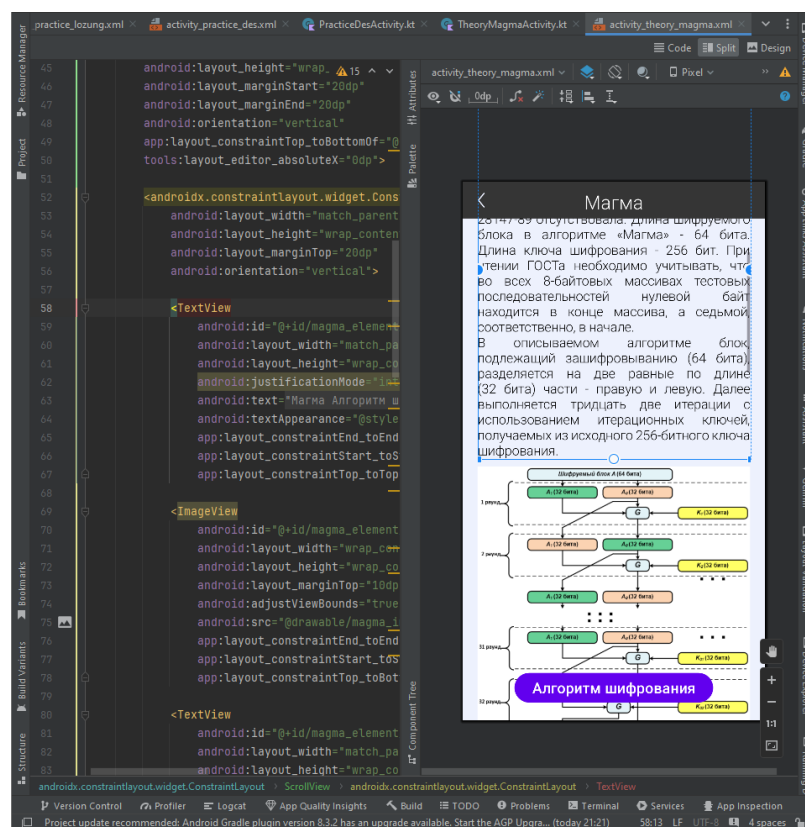
Листинг 4 – Главный экран приложения, которая написана разметкой XML, реализованный в Android Studio



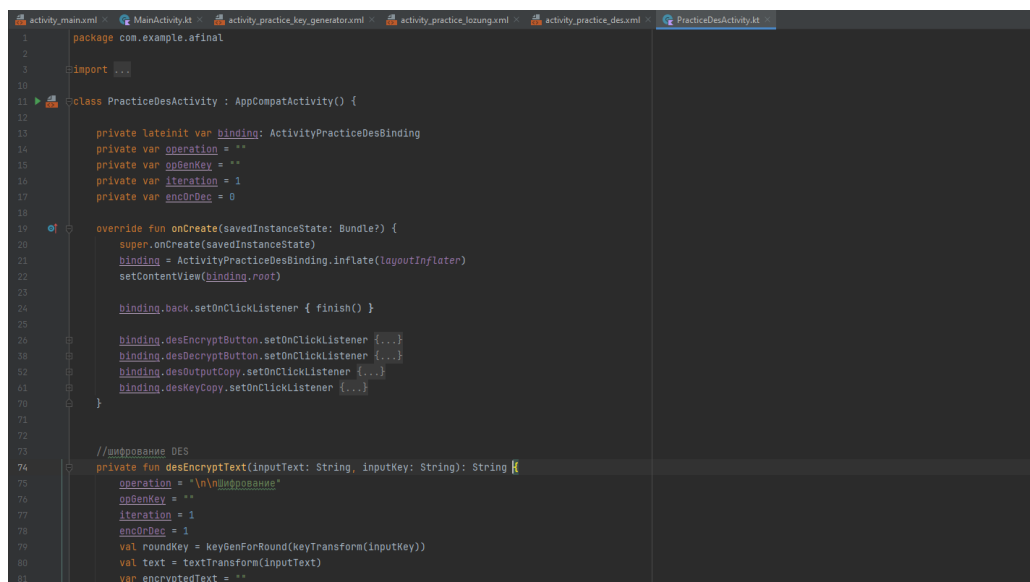
Листинг 5 – Главный экран приложения, логика главного экрана написана языком Kotlin, реализованный в Android Studio



Листинг 6 – Окно теории «Магма», которая написана разметкой XML, реализованный в Android Studio



Листинг 7 – Окно логики «DES», логика экрана «DES» написана языком Kotlin, реализованный в Android Studio



3.5 Инструкция пользования приложением

При запуске приложения открывается главный экран, см. рисунок 27. С главного можно перейти на 11 экранов с теоретическим описанием алгоритмов, а также на 12 экранов реализации алгоритмов шифрования.

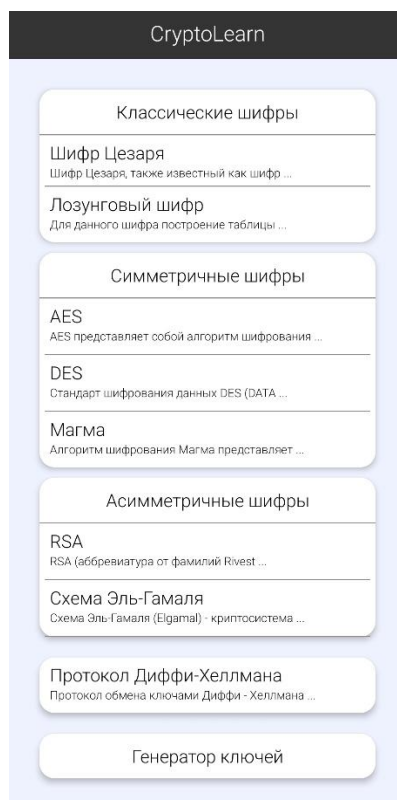


Рисунок 27 – Главный экран

Окна с теоретическим описанием алгоритма, следующие «Классические шифры», «Шифр Цезаря», «Лозунговый шифр», «Симметричные шифры», «AES», «DES», «Магма», «Асимметричные шифры», «RSA», «Схема Эль-Гамала» и «Протокол Диффи-Хеллмана», см. рисунок 28.

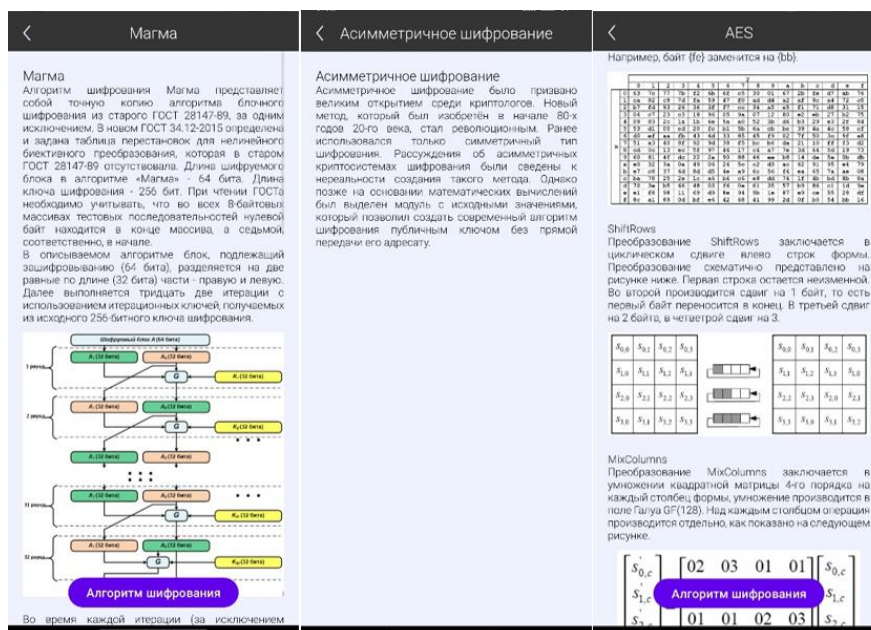


Рисунок 28 – Окна с теоретическим описанием алгоритмов

Окна с реализацией алгоритма шифрования, следующие «Классические шифры», «Шифр Цезаря», «Лозунговый шифр», «Симметричные шифры», «AES», «DES», «Магма», «Асимметричные шифры», «RSA», «Схема Эль-Гамала», «Протокол Диффи-Хеллмана» и «Генератор паролей», см. рисунок 29.

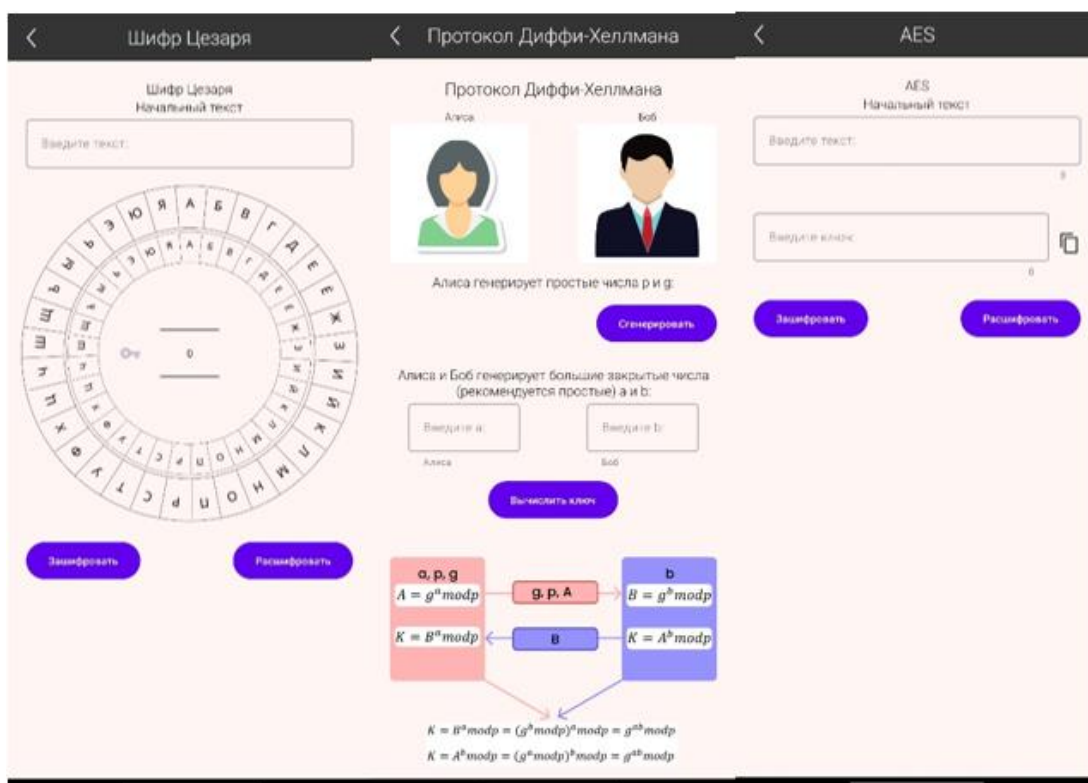


Рисунок 29 – Окна с реализацией алгоритма шифрования

С главного экрана одним нажатием открывается все окна с теоретическим описанием алгоритма и «Генератор паролей». Есть два варианта открыть окна с реализацией алгоритма шифрования:

1. Удерживая пункты «Шифр Цезаря», «Лозунговый шифр», «AES», «DES», «Магма», «RSA», «Схема Эль-Гамала», «Протокол Диффи-Хеллмана» и «Генератор паролей»
2. В окнах с теоретическим описанием «Шифр Цезаря», «Лозунговый шифр», «AES», «DES», «Магма», «RSA», «Схема Эль-Гамала», «Протокол Диффи-Хеллмана» снизу экрана закреплена кнопка «Алгоритм шифрования», данная кнопка открывает окна с реализацией алгоритма шифрования.

В каждом экране, кроме главной в левом верхнем углу расположена кнопка «Назад», она имеет форму '<'.

Инструкция пользования окон с реализацией алгоритма шифрования:

1. «Шифр Цезаря».

- 1.1. В данном окне необходимо ввести входные данные в поле ввода текста. Исходя из первой буквы введённых данных диск и длина ключа подстраиваются под необходимый расклад алфавита, т.е. если буквы диска на кириллице при введении латинского слова, буквы диска заменяются на латиницу, а длина ключа сократится на 25 и наоборот.
- 1.2. Ключ шифрования выбирается путём пролистывания счётчика.
- 1.3. Длина ключа зависит от алфавита и всегда подчиняется условию $n-1$, где n количество букв алфавита.
- 1.4. Поле вывода по умолчанию скрыто, оно появится при шифровании или дешифровании введённого текста, см. рисунок 30.
- 1.5. Также для удобства добавлена кнопка копирования конечного текста.

Рисунок 30 – Шифр Цезаря

2. «Лозунговый шифр».

2.1. В данном окне необходимо ввести входные данные в поле ввода текста и ключ в поле для ввода ключа. Исходя из первой буквы введенных данных исходная таблица и таблица замены будет подстроены под нужную раскладку алфавита.

2.2. Поле вывода по умолчанию скрыто, она появится при шифровании или дешифровании введенного текста, см. рисунок 31.

2.3. Также для удобства добавлена кнопка копирования конечного текста.

Рисунок 31 – Лозунговый шифр

3. «AES».

3.1.В данном окне необходимо ввести входные данные в поле ввода текста и ключ в поле для ввода ключа. Раскладка алфавита и длина входных данных не имеет значения.

3.2.Поле вывода по умолчанию скрыта, она появится при шифровании или дешифровании введенного текста, см. рисунок 32.

3.3.Также для удобства добавлены две кнопки копирования, для ключа и для конечного текста.

3.4.В поле «Алгоритм» будет отображена пошаговое вычисление генерации ключа для первого раунда и первый раунд.

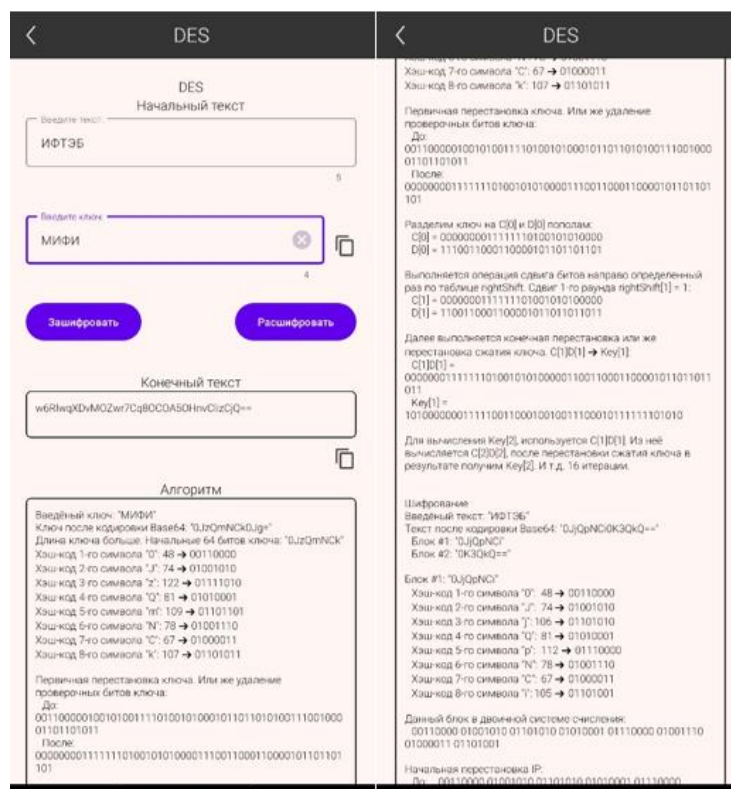


Рисунок 33 – DES

5. «Магма».

5.1. В данном окне необходимо ввести входные данные в поле ввода текста и ключ в поле для ввода ключа. Раскладка алфавита и длина входных данных не имеет значения.

5.2. Поле вывода по умолчанию скрыто, она появится при шифровании или дешифровании введенного текста, см. рисунок 34.

5.3. Также для удобства добавлены две кнопки копирования, для ключа и для конечного текста.

5.4. В поле «Алгоритм» будет отображена пошаговая вычисление генерации ключа для первого раунда и первый раунд.

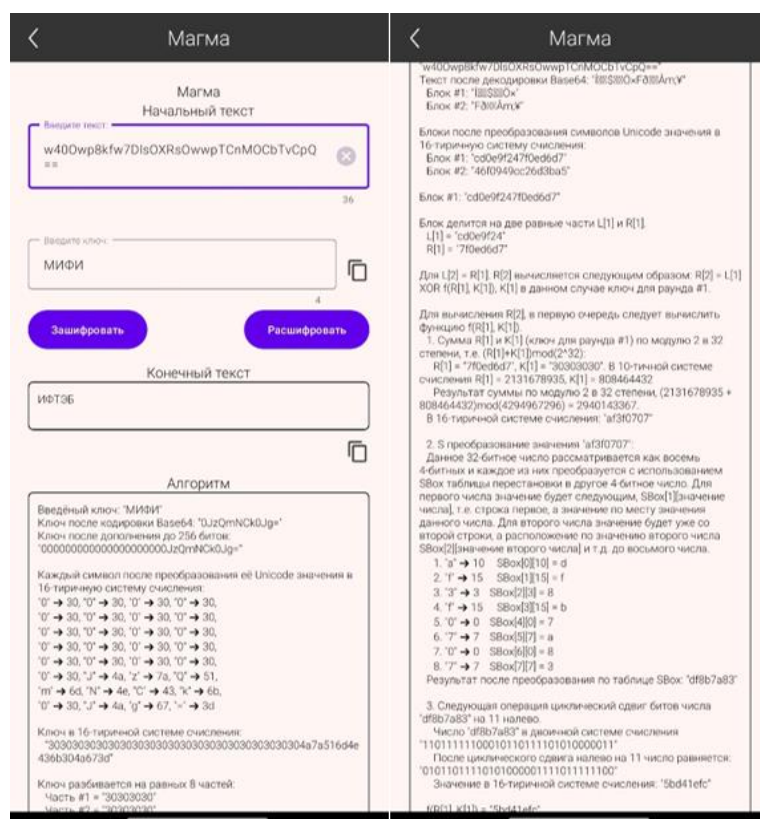


Рисунок 34 – Магма

6. «RSA».

- 6.1. Изначально необходимо сгенерировать ключи. Поле для вывода сгенерированных ключей по умолчанию скрыта. Указана подсказка указывающее на необходимость нажатия кнопки. Чтобы сгенерировать ключи необходимо нажать на кнопку «Сгенерировать ключи».
- 6.2. При необходимости можно очистить поле для вывода сгенерированных ключей нажав на кнопку «Очистить».
- 6.3. В данном окне необходимо ввести входные данные в поле ввода текста. Раскладка алфавита и длина входных данных не имеет значения. В поле «X» вводится значение «e» при шифровании и «d» при дешифровании. В обоих случаях в поле «N» вводится значение «n».
- 6.4. Поле вывода по умолчанию скрыта, она появится при шифровании или дешифровании введенного текста, см. рисунок 35.
- 6.5. Также для удобства добавлена кнопка копирования для конечного текста.

6.6.В поле «Алгоритм» будет отображена пошаговое вычисление.

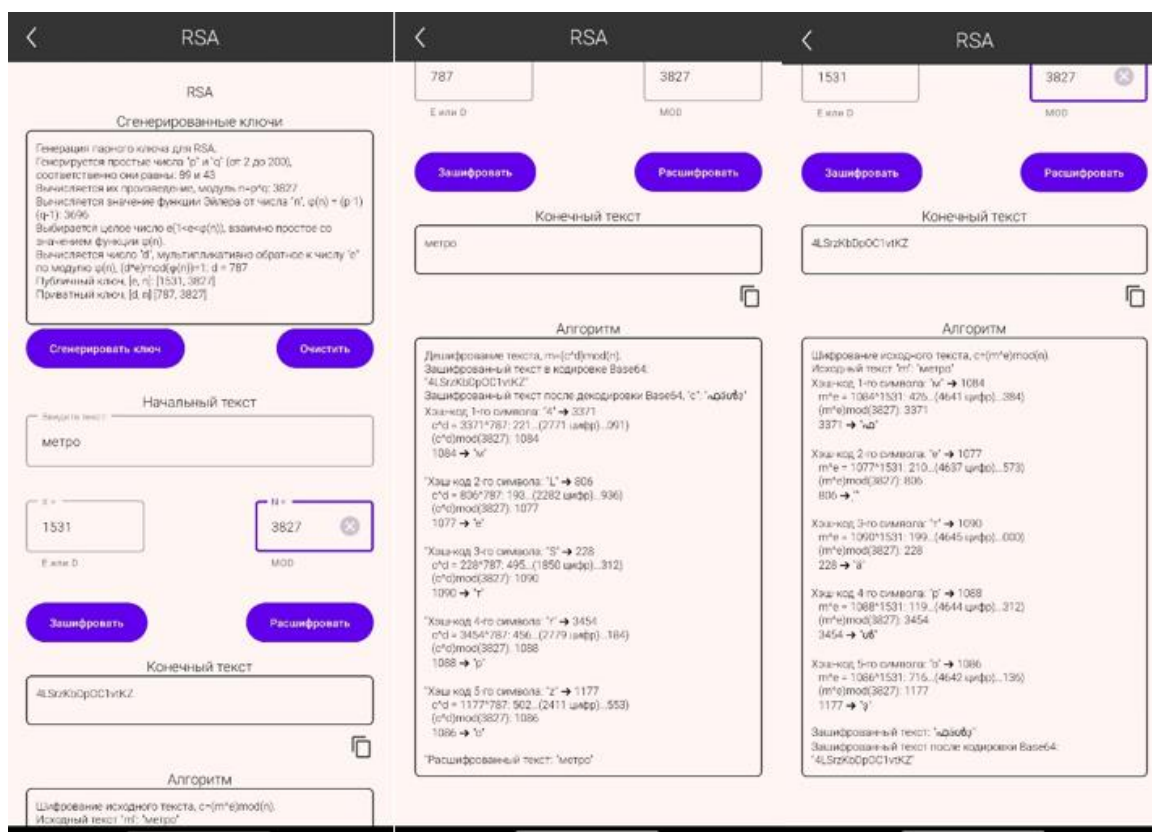


Рисунок 35 – RSA

7. «Схема Эль-Гамала».

7.1.Изначально необходимо сгенерировать ключи. Поле для вывода сгенерированных ключей по умолчанию скрыта. Указана подсказка указывающее на необходимость нажатия кнопки. Чтобы сгенерировать ключи необходимо нажать на кнопку «Сгенерировать ключи».

7.2.При необходимости можно очистить поле для вывода сгенерированных ключей нажав на кнопку «Очистить».

7.3.В данном окне необходимо ввести входные данные в поле ввода текста. Раскладка алфавита и длина входных данных не имеет значения. В поле и при шифровании, и при дешифровании «р» вводится значение «р», в поле «Z» при шифровании вводится значение «g» и «A» при дешифровании. В поле «K» при шифровании вводится значение «Y» и «x» при дешифровании.

7.4.Значение «А» вычисляется отправителем и отправляется вместе с шифротекстом.

7.5.Поле вывода по умолчанию скрыта, она появится при шифровании или дешифровании введенного текста, см. рисунок 36.

7.6.Также для удобства добавлена кнопка копирования для конечного текста.

7.7.В поле «Алгоритм» будет отображена пошаговое вычисление.

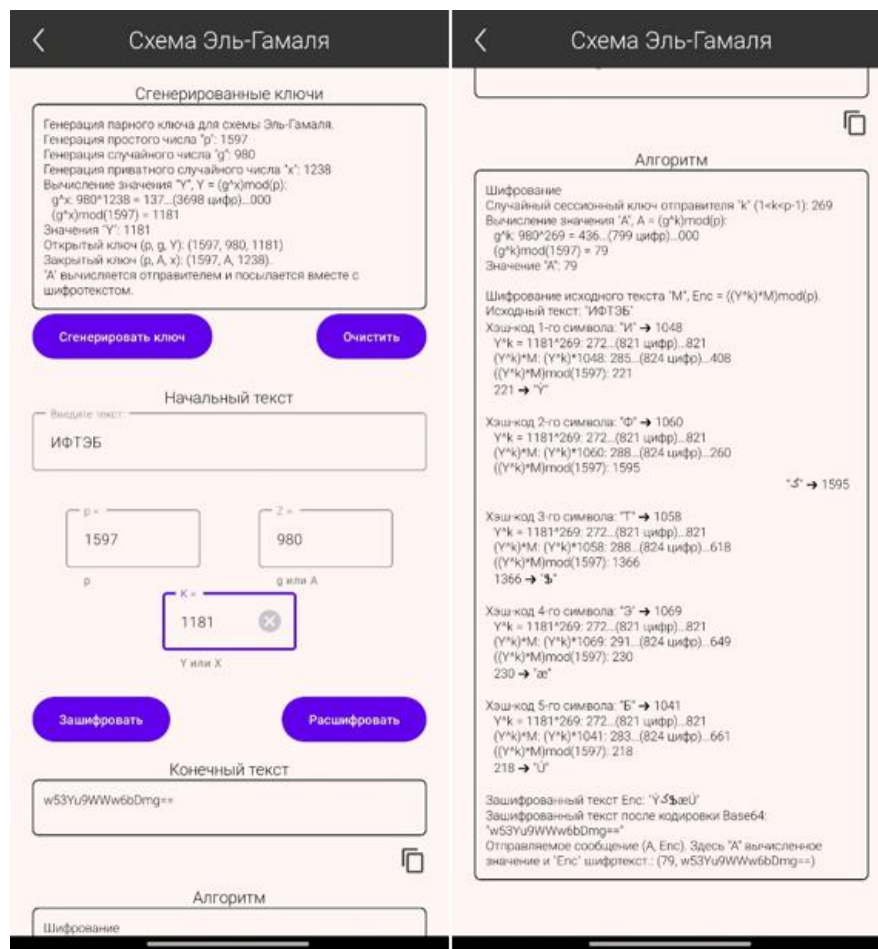


Рисунок 36 – Схема Эль-Гамала

8. «Протокол Диффи-Хеллмана».

8.1.Изначально необходимо сгенерировать «р» и «g». Поле для вывода сгенерированных чисел по умолчанию скрыта. Чтобы сгенерировать числа необходимо нажать на кнопку «Сгенерировать».

8.2.«р» и «g» используются для вычисления ключа, поэтому необходимо их сгенерировать.

8.3. Далее необходимо для полей ввода Алисы и Боб внести приватные числа Алисы и Боба. После необходимо нажать на кнопку «Вычислить ключ».

8.4. Под формулами появятся значения вычисления.

8.5. Поле вывода «Алгоритм» по умолчанию скрыта, она появится после нажатия на кнопку «Вычислить ключ» и будет отображена пошаговое вычисление, см. рисунок 37.

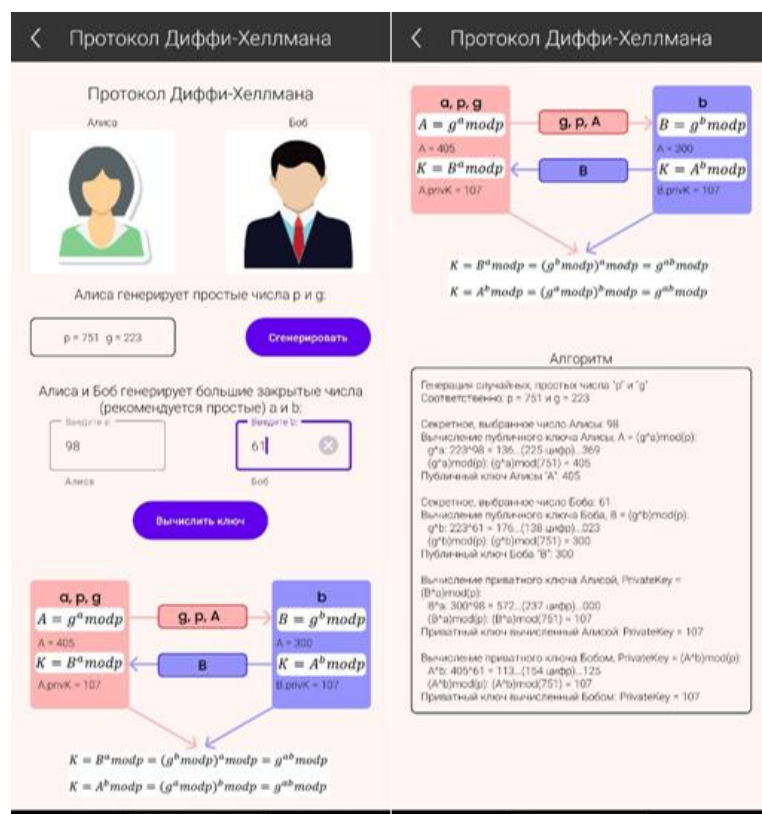


Рисунок 37 – Протокол обмена ключами Диффи-Хеллмана

9. «Генератор паролей».

9.1. Изначально необходимо из списка выбрать нужный набор символов для генерации.

9.2. Затем ввести желаемую длину пароля и нажать на кнопку «Сгенерировать» для генерации пароля.

9.3. Поле вывода по умолчанию скрыта, она появится после нажатия на кнопку «Сгенерировать».

9.4. При необходимости можно очистить поле вывода нажав на кнопку «Очистить».

9.5. Для удобства добавлена кнопка копирования сгенерированного ключа, см. рисунок 38.

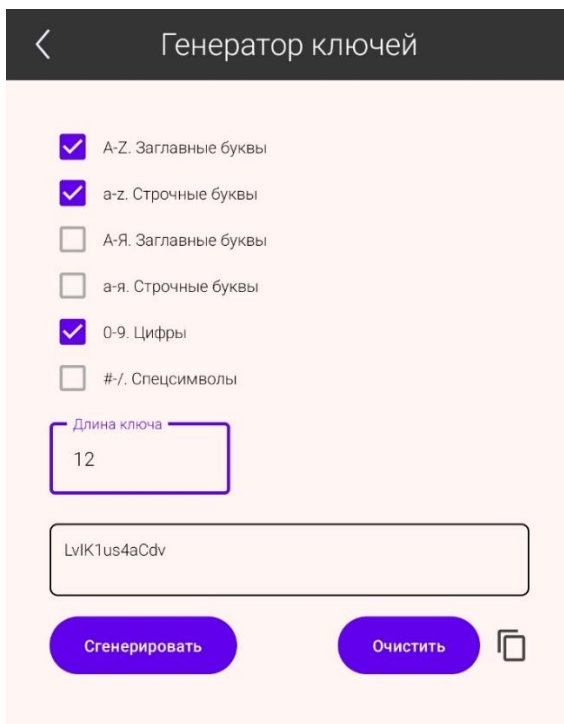


Рисунок 38 – Генератор паролей

ЗАКЛЮЧЕНИЕ

Разработка мобильного приложения для интерактивного обучения алгоритмам криптографического шифрования, проведенная в рамках данной работы, является актуальной и важной задачей в условиях растущих угроз информационной безопасности и необходимости защиты конфиденциальной информации.

В ходе проекта были достигнуты следующие результаты:

1. Обзор криптографических понятий и алгоритмов:
 - Проведен обзор основных понятий криптографии;
 - Рассмотрены как классические, так и современные алгоритмы шифрования, что позволило определить методы для реализации в приложении.
2. Обзор образовательных ресурсов:
 - Проанализированы существующие образовательные ресурсы по криптографии.
3. Сравнение мобильных приложений:
 - Проанализированы три существующих мобильных приложения: «Зашифровать Расшифровать», «Crypto - Encryption Tools» и «Enigma - Криптография и хэш». Это позволило выделить сильные и слабые стороны, и использовать полученные знания для разработки продукта.
4. Моделирование структуры приложения:
 - Разработана и смоделирована структура приложения, выбраны инструменты и технологии для разработки: язык программирования Kotlin, IntelliJ IDEA, Android Studio и JDK.
5. Процесс разработки:
 - Установлены и настроены необходимые программные инструменты.

- Разработаны и протестированы алгоритмы шифрования и дешифрования.
- Разработан удобный и интуитивно понятный интерфейс приложения.

Таким образом, выполненный проект позволил разработать мобильное приложение для интерактивного обучения алгоритмам криптографического шифрования для Android смартфонов, соответствующее требованиям и удобствам использования. Приложение включает в себе 8 различных криптографических алгоритмов. Полученные результаты могут быть использованы для дальнейшего совершенствования приложения и расширения его функционала, что обеспечит его актуальность и востребованность пока есть необходимость изучения алгоритмов шифрования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Васильева И. Н. Криптографические методы защиты информации. М.: Юрайт. 2024. 350 с.
2. Лось А. Б., Нестеренко А. Ю., Рожков М. И. Криптографические методы защиты информации для изучающих компьютерную безопасность. М.: Юрайт. 2024. 474 с.
3. Адаменко М. В. Основы классической криптологии. Секреты шифров и кодов. М.: ДМК Пресс. 2016. 296 с.
4. Бабаш А. В., Баранова Е. К., Ларин Д. А. Информационная безопасность. История защиты информации в России. М.: КДУ. 2017. 736 с.
5. Тарасов А.М. Криптография и электронная цифровая подпись // Российский следователь. 2014. N 1. С. 50 - 54; N 2. С. 45 - 49.
6. П. С. Шевчке, С.В.Соколов, С.О.Крамаров. Криптографическая защита информации: учебное пособие / Москва, 2019 - 321с. - (Высшее образование). - DOI:<https://doi.org/10.12737/1716-6>
7. <https://inclient.ru/smartphone-stats/#lwptoc> - ресурс для проверки статистики использования смартфонов по России [дата посещения - 17.01.2024 г.]
8. <https://www.statista.com/forecasts/1147055/smartphone-users-in-russia> - ресурс для проверки статистики использования смартфонов по России [дата посещения - 17.01.2024 г.]
9. https://rosstat.gov.ru/storage/mediabank/Ejegodnik_2023.pdf - Росстат - Федеральная служба государственной статистики [дата посещения - 17.01.2024 г.]
10. <https://tass.ru/obschestvo/9508331> - Тасс, Информационное агентство России ТАСС [дата посещения - 17.01.2024 г.]

- 11.<https://www.statista.com/topics/779/mobile-internet/#topicOverview> - ресурс для проверки статистики использования смартфонов по России [дата посещения - 17.01.2024 г.]
- 12.https://radar.yandex.ru/device_categories?period=2023-05-08%3A2024-01-17&chart_type=line2 - ресурс для проверки статистики использования смартфонов по России [дата посещения - 17.01.2024 г.]
- 13.<https://play.google.com/store/apps/details?id=com.socpay.textcrypter&pli=1> - мобильное приложение для Android смартфонов «Зашифровать Расшифровать» [дата просмотра - 6.02.2024 г.]
- 14.<https://play.google.com/store/apps/details?id=com.kokoschka.michael.crypto> - мобильное приложение для Android смартфонов «Crypto - Encryption Tools» [дата просмотра - 9.02.2024 г.]
15. <https://play.google.com/store/apps/details?id=app.enigma> - мобильное приложение для Android смартфонов «Enigma - Криптография и хэш» [дата просмотра - 11.02.2024 г.]
- 16.https://play.google.com/store/apps/details?id=duy.com.text_converter - мобильное приложение для Android смартфонов «Text Converter Encoder Decoder» [дата просмотра - 11.02.2024 г.]
- 17.<https://play.google.com/store/apps/details?id=com.astler.cipherscreateasecret> - мобильное приложение для Android смартфонов «Шифры! Морзе, Виженер, Цезарь» [дата просмотра - 12.02.2024 г.]
- 18.<https://play.google.com/store/apps/details?id=com.santodev.endcrypt> - мобильное приложение для Android смартфонов «Encrypt Decrypt Tools» [дата просмотра - 12.02.2024 г.]
- 19.<https://metanit.com/kotlin/tutorial/1.1.php> - Сайт о программировании [дата посещения - 16.02.2024 г.]
- 20.https://ru.ruwiki.ru/wiki/Android_Studio - Интернет энциклопедия [дата посещения - 16.02.2024 г.]

- 21.https://ru.wikipedia.org/wiki/Java_Development_Kit - Интернет энциклопедия [дата посещения - 06.04.2024 г.]
- 22.<https://skillbox.ru/media/code/yazyk-programmirovaniya-kotlin/> - Образовательная платформа с онлайн курсами [дата посещения - 06.04.2024 г.]
- 23.<https://www.jetbrains.com/ru-ru/idea/> - Разработчик программного обеспечения [дата посещения - 07.04.2024 г.]
- 24.<https://skillbox.ru/media/code/chto-takoe-android-studio-i-kak-ey-polzovatsya/> - Образовательная платформа с онлайн курсами [дата посещения - 07.04.2024 г.]
- 25.https://ru.wikipedia.org/wiki/Шифр_Цезаря - Интернет энциклопедия [дата посещения - 08.04.2024 г.]
- 26.<https://nightquests.ru/knowledgebases/lozungovyy-shifr/> - Интернет ресурс [дата посещения - 09.04.2024 г.]
- 27.<https://csrc.nist.gov/external/nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf> - NIST, документация алгоритма симметричного шифрования AES [даты просмотра и изучения 10.04.2024-15.04.2024 гг.]
- 28.<https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf> - NIST, документация алгоритма симметричного шифрования DES [даты просмотра и изучения 15.04.2024-19.04.2024 гг.]
- 29.<https://rst.gov.ru:8443/file-service/file/load/1699367634477> - Росстандарт, документация алгоритма симметричного шифрования Магма [даты просмотра и изучения 19.04.2024-24.04.2024 гг.]
- 30.<https://cetinkayakoc.net/docs/r01.pdf> - Документация алгоритма асимметричного шифрования RSA [даты просмотра и изучения 24.04.2024-27.04.2024 гг.]
- 31.<https://homepages.math.uic.edu/~leon/mcs425-s08/handouts/el-gamal.pdf> - Документация алгоритма асимметричного шифрования Схемы Эль-Гамала [даты просмотра и изучения 27.04.2024-30.04.2024 гг.]

- 32.<https://www.meity.gov.in/writereaddata/files/Diffie-Hellman.pdf> - Документация протокола обмена ключами Диффи-Хеллмана [даты просмотра и изучения 01.05.2024-03.05.2024 гг.]
- 33.<https://www.youtube.com/> - Платформа для просмотра видеороликов. [даты посещения 10.04.2024-03.05.2024 гг.]

ПРИЛОЖЕНИЕ А

Алгоритм шифрования «Шифр Цезаря»

```
import java.util.*
private val sc = Scanner(System.`in`)
//основной метод
fun main(){
    println(caesarEncrypt())
    println(caesarDecrypt())
}
//шифрование
fun caesarEncrypt(): String {
    val text = sc.nextLine()
    var encText = ""
    var key = sc.nextInt()
    key %= alphabetCaesar(if(text[0].hashCode()<128){1}else{0}).length
    val bigAlph = alphabetCaesar(if(text[0].hashCode()<128){1}else{0})
    val smallAlph = bigAlph.lowercase()
    val changeBigAlph = bigAlph.substring(key) + bigAlph.substring(0,key)
    val changeSmallAlph = changeBigAlph.lowercase()
    var sym = false
    for(i in text.indices){
        for(j in bigAlph.indices){
            if(text[i]==bigAlph[j]){
                encText+=changeBigAlph[j]
                sym = true
            }
            if(text[i]==smallAlph[j]){
                encText+=changeSmallAlph[j]
                sym = true
            }
        }
        if(!sym){
            encText+=text[i]
        }
        sym=false
    }
    return encText
}
//дешифрование
fun caesarDecrypt(): String {
    val text = sc.nextLine()
    var decText = ""
    var key = sc.nextInt()
    key %= alphabetCaesar(if(text[0].hashCode()<128){1}else{0}).length
    val bigAlph = alphabetCaesar(if(text[0].hashCode()<128){1}else{0})
    val smallAlph = bigAlph.lowercase()
    val changeBigAlph = bigAlph.substring(key) + bigAlph.substring(0,key)
    val changeSmallAlph = changeBigAlph.lowercase()
    var sym = false
    for(i in text.indices){
        for(j in bigAlph.indices){
            if(text[i]==changeBigAlph[j]){
                decText+=bigAlph[j]
                sym = true
            }
        }
        if(text[i]==changeSmallAlph[j]){
            decText+=smallAlph[j]
        }
    }
}
```

```

        sym = true
    }
}
if(!sym){
    decText+=text[i]
}
sym=false
}
return decText
}
//функция алфвита
fun alphabetCaesar(a: Int): String {
    val kiril = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    val latin = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    return if(a == 1){latin}else{kiril}
}

```

ПРИЛОЖЕНИЕ Б

Алгоритм шифрования «Лозунговый шифр»

```
import java.util.*
private val sc = Scanner(System.`in`)
//основной метод
fun main(){
    println(lozungEncrypt())
    //    println(lozungDecrypt())
}
//шифрование
fun lozungEncrypt(): String {
    val text = sc.nextLine()
    val key = sc.next()
    var encText = ""
    val bigAlph = alphabetLozung(
        if (key[0].hashCode() < 128) {
            1
        } else {
            0
        }
    )
    val smallAplh = bigAlph.lowercase()
    val changeBigAlph = remove(key)
    val changeSmallALph = changeBigAlph.lowercase()
    if (if (key[0].hashCode() < 128) {
        1
    } else {
        0
    } == 1) {
        ) {
            var sym = false
            for (i in text.indices) {
                for (j in bigAlph.indices) {
                    if (text[i] == bigAlph[j]) {
                        encText += changeBigAlph[j]
                        sym = true
                    }
                    if (text[i] == smallAplh[j]) {
                        encText += changeSmallALph[j]
                        sym = true
                    }
                }
                if (!sym) {
                    encText += text[i]
                }
                sym = false
            }
        }
        return encText
    }
}
//дешифрование
fun lozungDecrypt(): String {
    val text = sc.nextLine()
    val key = sc.next()
    var decText = ""
    val bigAlph = alphabetLozung(if (key[0].hashCode() < 128) {1} else {0})
    val smallAlph = bigAlph.lowercase()
```

```

val changeBigAlpha = remove(key)
val changeSmanllAlpha = changeBigAlpha.lowercase()
var sym = false
for(i in text.indices){
    for(j in bigAlpha.indices){
        if(text[i]==changeBigAlpha[j]){
            decText+=bigAlpha[j]
            sym = true
        }
        if(text[i]==changeSmanllAlpha[j]){
            decText+=smallAlpha[j]
            sym = true
        }
    }
    if(!sym){
        decText+=text[i]
    }
    sym=false
}
return decText
}
//обработка ключа
private fun remove(key: String): String {
    var k = key.uppercase()
    val x = (k.toMutableList() + alphabetLo-
zung(if(key[0].hashCode()<128){1}else{0}).toMutableList()).distinct()
    k=""
    for(i in alphabetLozung(if(key[0].hashCode()<128){1}else{0}).indices){
        k+=x[i].toString()
    }
    return k
}
fun alphabetLozung(a: Int): String {
    val kiril = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"
    val latin = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    return if(a == 1){latin}else{kiril}
}

```

ПРИЛОЖЕНИЕ В

Алгоритм шифрования «AES»

```
import java.util.*
private val sc = Scanner(System.`in`)
//основной метод
fun main() {
    println(aesEncryptText())
    println(aesDecryptText())
}
//алгоритм шифрования
private fun aesEncryptText(): String {
    var encryptedText = ""
    val text = textTransform(sc.nextLine(), 1)
    val keyRound = Array(11) { Array(4) { Array(4) { 0 } } }
    for (r in 0..10) {
        if (r == 0) {
            keyRound[r] = keyTransform(sc.nextLine())
        } else {
            keyRound[r] = keyGenForRound(keyRound[r - 1], r)
        }
    }
    for (block in text.indices) {
        val tx = text[block]
        var encText = Array(4) { Array(4) { 0 } }
        val state = Array(4) { Array(4) { 0 } }
        var z = 0
        for (i in 0..3) {
            for (j in 0..3) {
                state[j][i] = tx[z].hashCode()
                z++
            }
        }
        // Нулевого раунда
        for (i in 0..3) {
            for (j in 0..3) {
                encText[i][j] = keyRound[0][i][j] xor state[i][j]
            }
        }
        // начало 10ти раундов
        var round = 1
        for (i in 0..9) {
            encText = subBytes(encText, block)
            encText = shiftRows(encText, block)
            if (i < 9) {
                encText = mixColumns(encText, block)
            }
            // AddRoundKey
            for (j in 0..3) {
                for (k in 0..3) {
                    encText[j][k] = keyRound[round][j][k] xor encText[j][k]
                }
            }
            round++
        }
        var kText = ""
        for (i in 0..3) {
            for (j in 0..3) {
```

```

        kText += encText[j][i].toChar()
    }
    encryptedText += kText
}
return encryptedText.base64Encode()
}
/*алгоритм дешифрования*/
private fun aesDecryptText(): String {
    var decryptedText = ""
    val text = textTransform(sc.nextLine(), 2)
    val keyRound = Array(11) { Array(4) { Array(4) { 0 } } }
    for (r in 0..10) {
        if (r == 0) {
            keyRound[r] = keyTransform(sc.nextLine())
        } else {
            keyRound[r] = keyGenForRound(keyRound[r - 1], r)
        }
    }
    for (block in text.indices) {
        val tx = text[block]
        var decText = Array(4) { Array(4) { 0 } }
        val state = Array(4) { Array(4) { 0 } }
        var z = 0
        for (i in 0..3) {
            for (j in 0..3) {
                state[j][i] = tx[z].hashCode()
                z++
            }
        }
        // Нулевого раунда
        for (i in 0..3) {
            for (j in 0..3) {
                decText[i][j] = keyRound[10][i][j] xor state[i][j]
            }
        }
        // начало 10ти раундов
        var round = 1
        for (i in 0..9) {
            decText = invShiftRows(decText, block)
            decText = invSubBytes(decText, block)
            // AddRoundKey
            for (j in 0..3) {
                for (k in 0..3) {
                    decText[j][k] = keyRound[10-round][j][k] xor
decText[j][k]
                }
            }
            if (i < 9) {
                decText = invMixColumns(decText, block)
            }
            round++
        }
        var kText = ""
        for (i in 0..3) {
            for (j in 0..3) {
                kText += decText[j][i].toChar()
            }
        }
        decryptedText += kText
    }
}

```

```

        decryptedText = decryptedText.trim()
        return decryptedText.base64Decode()
    }
    // Перестановка Байтов с SBox таблицы
    private fun subBytes(encText: Array<Array<Int>>, block: Int): Ar-
    ray<Array<Int>> {
        var x = 0
        var y = 0
        for (i in 0..3) {
            for (j in 0..3) {
                if (encText[i][j].toString(16).length == 1) {
                    x = 0
                    when(encText[i][j].toString(16)[0]) {
                        '0' -> y=0
                        '1' -> y=1
                        '2' -> y=2
                        '3' -> y=3
                        '4' -> y=4
                        '5' -> y=5
                        '6' -> y=6
                        '7' -> y=7
                        '8' -> y=8
                        '9' -> y=9
                        'a' -> y=10
                        'b' -> y=11
                        'c' -> y=12
                        'd' -> y=13
                        'e' -> y=14
                        'f' -> y=15
                    }
                }
                if (encText[i][j].toString(16).length == 2) {
                    when(encText[i][j].toString(16)[0]) {
                        '0' -> x=0
                        '1' -> x=1
                        '2' -> x=2
                        '3' -> x=3
                        '4' -> x=4
                        '5' -> x=5
                        '6' -> x=6
                        '7' -> x=7
                        '8' -> x=8
                        '9' -> x=9
                        'a' -> x=10
                        'b' -> x=11
                        'c' -> x=12
                        'd' -> x=13
                        'e' -> x=14
                        'f' -> x=15
                    }
                }
                when(encText[i][j].toString(16)[1]) {
                    '0' -> y=0
                    '1' -> y=1
                    '2' -> y=2
                    '3' -> y=3
                    '4' -> y=4
                    '5' -> y=5
                    '6' -> y=6
                    '7' -> y=7
                    '8' -> y=8
                    '9' -> y=9
                }
            }
        }
    }

```



```

        'a' -> y=10
        'b' -> y=11
        'c' -> y=12
        'd' -> y=13
        'e' -> y=14
        'f' -> y=15
    }
}
encText[i][j] = sBox()[x][y]
}
}
return encText
}
// Перемешивание строк
private fun shiftRows(encText: Array<Array<Int>>, block: Int): Ar-
ray<Array<Int>> {
    val a = encText[1][0]
    encText[1][0] = encText[1][1]
    encText[1][1] = encText[1][2]
    encText[1][2] = encText[1][3]
    encText[1][3] = a
    val b = encText[2][0]
    val c = encText[2][1]
    encText[2][0] = encText[2][2]
    encText[2][1] = encText[2][3]
    encText[2][2] = b
    encText[2][3] = c
    val d = encText[3][3]
    encText[3][3] = encText[3][2]
    encText[3][2] = encText[3][1]
    encText[3][1] = encText[3][0]
    encText[3][0] = d
    return encText
}
// Преобразование MixColumns
private fun mixColumns(encText: Array<Array<Int>>, block: Int): Ar-
ray<Array<Int>> {
    val kText = Array(4) { Array(4) { 0 } }
    val mC = arrayOf(
        arrayOf(0x02, 0x03, 0x01, 0x01),
        arrayOf(0x01, 0x02, 0x03, 0x01),
        arrayOf(0x01, 0x01, 0x02, 0x03),
        arrayOf(0x03, 0x01, 0x01, 0x02)
    )
    for(i in 0..3){
        for(j in 0..3){
            for(k in 0..3){
                kText[i][j] = gMul(encText[k][j], mC[i][k]) xor kText[i][j]
            }
        }
    }
    return kText
}
//массив S-Box
private fun sBox(): Array<Array<Int>> {
    val array = arrayOf(
        0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67,
        0x2b, 0xfe, 0xd7, 0xab, 0x76,
        0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2,
        0xaf, 0x9c, 0xa4, 0x72, 0xc0,
        0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5,

```

```

0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80,
0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6,
0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe,
0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02,
0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda,
0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e,
0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8,
0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac,
0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4,
0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74,
0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57,
0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87,
0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d,
0x0f, 0xb0, 0x54, 0xbb, 0x16
    )
    val sBox = Array(16) { Array(16) { 0 } }
    var k = 0
    for (i in 0..15) {
        for (j in 0..15) {
            sBox[i][j] = array[k]
            k++
        }
    }
    return sBox
}
/*массив inv S-Box, обратный S-Box*/
private fun invSBox(): Array<Array<Int>> {
    val array = arrayOf(
        0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3,
0x9e, 0x81, 0xf3, 0xd7, 0xfb,
        0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43,
0x44, 0xc4, 0xde, 0xe9, 0xcb,
        0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95,
0x0b, 0x42, 0xfa, 0xc3, 0x4e,
        0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2,
0x49, 0x6d, 0x8b, 0xd1, 0x25,
        0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c,
0xcc, 0x5d, 0x65, 0xb6, 0x92,
        0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46,
0x57, 0xa7, 0x8d, 0x9d, 0x84,
        0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58,
0x05, 0xb8, 0xb3, 0x45, 0x06,
        0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd,
0x03, 0x01, 0x13, 0x8a, 0x6b,
        0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf,
0xce, 0xf0, 0xb4, 0xe6, 0x73,
        0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37,
0xe8, 0x1c, 0x75, 0xdf, 0x6e,

```

```

        0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62,
0x0e, 0xaa, 0x18, 0xbe, 0x1b,
        0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0,
0xfe, 0x78, 0xcd, 0x5a, 0xf4,
        0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10,
0x59, 0x27, 0x80, 0xec, 0x5f,
        0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a,
0x9f, 0x93, 0xc9, 0x9c, 0xef,
        0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb,
0x3c, 0x83, 0x53, 0x99, 0x61,
        0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14,
0x63, 0x55, 0x21, 0x0c, 0x7d
    )
    val invSBox = Array(16) { Array(16) { 0 } }
    var k = 0
    for (i in 0..15) {
        for (j in 0..15) {
            invSBox[i][j] = array[k]
            k++
        }
    }
    return invSBox
}

// Перемешивание строк invShiftRows
private fun invShiftRows(decText: Array<Array<Int>>, block: Int): Ar-
ray<Array<Int>> {
    val a = decText[1][3]
    decText[1][3] = decText[1][2]
    decText[1][2] = decText[1][1]
    decText[1][1] = decText[1][0]
    decText[1][0] = a
    val b = decText[2][0]
    val c = decText[2][1]
    decText[2][0] = decText[2][2]
    decText[2][1] = decText[2][3]
    decText[2][2] = b
    decText[2][3] = c
    val d = decText[3][0]
    decText[3][0] = decText[3][1]
    decText[3][1] = decText[3][2]
    decText[3][2] = decText[3][3]
    decText[3][3] = d
    return decText
}

// Перестановка Байтов с SBox таблицы
private fun invSubBytes(decText: Array<Array<Int>>, block: Int): Ar-
ray<Array<Int>> {
    var x = 0
    var y = 0
    for (i in 0..3) {
        for (j in 0..3) {
            if (decText[i][j].toString(16).length == 1) {
                x = 0
                when (decText[i][j].toString(16)[0]) {
                    '0' -> y=0
                    '1' -> y=1
                    '2' -> y=2
                    '3' -> y=3
                    '4' -> y=4
                    '5' -> y=5
                    '6' -> y=6

```

```

        '7' -> y=7
        '8' -> y=8
        '9' -> y=9
        'a' -> y=10
        'b' -> y=11
        'c' -> y=12
        'd' -> y=13
        'e' -> y=14
        'f' -> y=15
    }
}
if (decText[i][j].toString(16).length == 2) {
    when(decText[i][j].toString(16)[0]) {
        '0' -> x=0
        '1' -> x=1
        '2' -> x=2
        '3' -> x=3
        '4' -> x=4
        '5' -> x=5
        '6' -> x=6
        '7' -> x=7
        '8' -> x=8
        '9' -> x=9
        'a' -> x=10
        'b' -> x=11
        'c' -> x=12
        'd' -> x=13
        'e' -> x=14
        'f' -> x=15
    }
    when(decText[i][j].toString(16)[1]) {
        '0' -> y=0
        '1' -> y=1
        '2' -> y=2
        '3' -> y=3
        '4' -> y=4
        '5' -> y=5
        '6' -> y=6
        '7' -> y=7
        '8' -> y=8
        '9' -> y=9
        'a' -> y=10
        'b' -> y=11
        'c' -> y=12
        'd' -> y=13
        'e' -> y=14
        'f' -> y=15
    }
}
decText[i][j] = invSBox()[x][y]
}
}
return decText
}
//Преобразование invMixColumns
private fun invMixColumns(decText: Array<Array<Int>>, block: Int): Array<Array<Int>> {
    val kText = Array(4) { Array(4) { 0 } }
    val mC = arrayOf(
        arrayOf(0x0e, 0x0b, 0x0d, 0x09),
        arrayOf(0x09, 0x0e, 0x0b, 0x0d),

```

```

        arrayOf(0x0d, 0x09, 0x0e, 0x0b),
        arrayOf(0x0b, 0x0d, 0x09, 0x0e)
    )
    for(i in 0..3){
        for(j in 0..3){
            for(k in 0..3){
                kText[i][j] = gMul(decText[k][j], mC[i][k]) xor kText[i][j]
            }
        }
    }
    return kText
}
/*массив RCon*/
private fun rCon(): Array<Array<Int>> {
    val array = arrayOf(
        0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80, 0x1B, 0x36,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00)
    val rCon = Array(4) { Array(10) { 0x00 } }
    var k = 0
    for (i in 0..<4) {
        for (j in 0..<10) {
            rCon[i][j] = array[k]
            k++
        }
    }
    return rCon
}
//генерация ключей раундов
private fun keyGenForRound(key: Array<Array<Int>>, round: Int): Ar-
ray<Array<Int>> {
    val a = mutableListOf(key[1][3], key[2][3], key[3][3], key[0][3])
    var x = 0
    var y = 0
    for (i in 0..3) {
        if (a[i].toString(16).length == 1) {
            x = 0
            if (a[i].toString(16)[0].isDigit()) {
                y = a[i].toString(16)[0].toString().toInt()
            }
            if (a[i].toString(16)[0].isLetter()) {
                if (a[i].toString(16)[0] == 'a') {
                    y = 10
                }
                if (a[i].toString(16)[0] == 'b') {
                    y = 11
                }
                if (a[i].toString(16)[0] == 'c') {
                    y = 12
                }
                if (a[i].toString(16)[0] == 'd') {
                    y = 13
                }
                if (a[i].toString(16)[0] == 'e') {
                    y = 14
                }
                if (a[i].toString(16)[0] == 'f') {
                    y = 15
                }
            }
        }
    }
}

```

```

    }
    if (a[i].toString(16).length == 2) {
        if (a[i].toString(16)[0].isDigit()) {
            x = a[i].toString(16)[0].toString().toInt()
        }
        if (a[i].toString(16)[0].isLetter()) {
            if (a[i].toString(16)[0] == 'a') {
                x = 10
            }
            if (a[i].toString(16)[0] == 'b') {
                x = 11
            }
            if (a[i].toString(16)[0] == 'c') {
                x = 12
            }
            if (a[i].toString(16)[0] == 'd') {
                x = 13
            }
            if (a[i].toString(16)[0] == 'e') {
                x = 14
            }
            if (a[i].toString(16)[0] == 'f') {
                x = 15
            }
        }
        if (a[i].toString(16)[1].isDigit()) {
            y = a[i].toString(16)[1].toString().toInt()
        }
        if (a[i].toString(16)[1].isLetter()) {
            if (a[i].toString(16)[1] == 'a') {
                y = 10
            }
            if (a[i].toString(16)[1] == 'b') {
                y = 11
            }
            if (a[i].toString(16)[1] == 'c') {
                y = 12
            }
            if (a[i].toString(16)[1] == 'd') {
                y = 13
            }
            if (a[i].toString(16)[1] == 'e') {
                y = 14
            }
            if (a[i].toString(16)[1] == 'f') {
                y = 15
            }
        }
    }
    i
    a[i] = sBox()[x][y]
}
val keyGenRound = Array(4) { Array(4) { 0 } }
for(i in 0..3){
    for(j in 0..3){
        if (j==0) {
            keyGenRound[i][j] = key[i][j] xor a[i] xor rCon()[i][round-1]
        }else{
            keyGenRound[i][j] = key[i][j] xor keyGenRound[i][j - 1]
        }
    }
}

```

```

    }
    return keyGenRound
}
//GF Galois Field
private fun gMul(a: Int, b: Int): Int {
    var tmpA = a
    var tmpB = b
    var p = 0
    var hiBitSet: Int
    for (i in 0..<8) {
        if ((tmpB and 0x01) == 1) {
            p = (p xor tmpA) and 0xff
        }
        hiBitSet = tmpA and 0x80
        tmpA = tmpA shl 1
        if (hiBitSet == 0x80) {
            tmpA = tmpA xor 0x1b
        }
        tmpB = (tmpB shr 1) and 0x7f
    }
    return p
}
//Преобразование текста для шифрования
private fun textTransform(text: String, endOrDec: Int): Array<String> {
    var txt = ""
    if(endOrDec==1) {
        txt = text.base64Encode()
        if(txt.length%16!=0){
            repeat(16-txt.length%16){txt += " "}
        }
    }
    if(endOrDec==2) {
        txt = text.base64Decode()
    }
    val partedText = txt.partitioner(txt.length/16, 16)
    repeat(partedText.size){
    }
    return partedText
}
//Преобразование ключа
private fun keyTransform(key: String): Array<Array<Int>> {
    //    ограничить длину ключа максимум на 12 Анг или 8 кириллица
    var k = key.base64Encode()
    if(k.length>16){
        k = k.substring(0, 16)
    }else if(k.length<16){
        repeat(16-k.length%16){k = "0$k"}
    }
    val endKeyValue = Array(4) { Array( 4) { 0 } }
    var x = 0
    for (i in 0..3){
        for (j in 0..3) {
            endKeyValue[j][i] = k[x].hashCode()
            x++
        }
    }
    return endKeyValue
}
//partitioner extension function для деления текста на части с указанной дли-
ной// Возвращает Array<String>
private fun String.partitioner(arraySize: Int, partLength: Int): Ar-
```

```

ray<String> {
    return Array(arraySize) { i ->
        this.substring(i * partLength, (i + 1) * partLength)
    }
}
//extensions functions для кодировки и декодировки// Возвращает String
private fun String.base64Encode(): String { return
Base64.getEncoder().encodeToString(this.toByteArray()) }
private fun String.base64Decode(): String { return
String(Base64.getDecoder().decode(this)) }

```


ПРИЛОЖЕНИЕ Г

Алгоритм шифрования «DES»

```
import java.util.*
private val sc = Scanner(System.`in`)
//основной метод
fun main() {
    desEncryptText()
    desDecryptText()
}
//шифрование DES
fun desEncryptText(): String {
    val roundKey = keyGenForRound(keyTransform("kanat"))
    val text = textTransform("kanat parakhatov", 1)
    var encryptedText = ""
    for(block in text.indices){
        val tx = text[block]
        var txt = ""
        var z = ""
        for (i in tx.indices){
            var a = tx[i].hashCode().toString(2)
            while (a.length<8){
                a = "0$a"
            }
            txt += a
            z += "$a "
        }
        var encText = ""
        z = ""
        for(i in 0..63){
            encText += txt[IP[i]-1]
            z += if(i%8==0){
                "$ {txt[IP[i]-1]}"
            }else{
                "$ {txt[IP[i]-1]}"
            }
        }
        val l = Array(17) {""}
        val r = Array(17) {""}
        l[0] = encText.substring(0,32)
        r[0] = encText.substring(32)
        for(i in 1..16){
            l[i] = r[i-1]
            r[i] = isk(l[i - 1], f(r[i - 1], roundKey[i - 1], block))
        }
        encText = ""
        z = ""
        for(i in 0..63){
            encText += (r[16]+l[16])[IP_1[i]-1]
            if(i%8==0 && i!=0){
                z += " $ {(r[16]+l[16])[IP[i]-1]}"
            }else{
                z += "$ {(r[16]+l[16])[IP[i]-1]}"
            }
        }
        var kText = ""
        for(i in 0..7){
            kText += ("${encText.substring(i * 8, i * 8 +
```

```

8).toInt(2).toChar() }")
    }
    encryptedText += kText
}
return encryptedText.base64Encode()
}
//дешифрование DES
fun desDecryptText(): String {
    val roundKey = keyGenForRound(keyTransform("kanat"))
    val text = textTransform(
form("L3vDmlPDg8Kkw6jDhsKqw4wPWVE7bFPDnkEEw505w45vw6k=", 2)
    var decryptedText = ""
    for(block in text.indices){
        val tx = text[block]
        var txt = ""
        var z = ""
        for (i in tx.indices){
            var a = tx[i].hashCode().toString(2)
            while (a.length<8){
                a = "0$a"
            }
            txt += a
            z += "$a "
        }
        var decText = ""
        z = ""
        for(i in 0..63){
            decText += txt[IP[i]-1]
            z += if(i%8==0){
                " ${txt[IP[i]-1]}"
            }else{
                "${txt[IP[i]-1]}"
            }
        }
        val l = Array(17) {""}
        val r = Array(17) {""}
        l[0] = decText.substring(0,32)
        r[0] = decText.substring(32)
        for(i in 1..16){
            l[i] = r[i-1]
            r[i] = isk(l[i - 1], f(r[i - 1], roundKey[16 - i], block ))
        }
        decText = ""
        z = ""
        for(i in 0..63){
            decText += (r[16]+l[16])[IP_1[i]-1]
            if(i%8==0 && i!=0){
                z += " ${(r[16]+l[16])[IP[i]-1]}"
            }else{
                z += "${(r[16]+l[16])[IP[i]-1]}"
            }
        }
        var kText = ""
        for(i in 0..7){
            kText += ("${decText.substring(i * 8, i * 8 +
8).toInt(2).toChar() }")
        }
        decryptedText += kText
    }
    decryptedText = decryptedText.trim()
    return decryptedText.base64Decode()
}

```

```

}
//функция f(r, k)
private fun f(r: String, key: String, block: Int): String {
    var a = ""
    for (i in 0..47) {
        a += r[e_R[i] - 1]
    }
    return sb(isk(a, key), block)
}
//операция SBox
private fun sb(txt: String, block: Int): String {
    val nBox = Array(8) { "" }
    var z = ""
    for (i in 0..7) {
        nBox[i] = txt.substring(i*6, i*6+6)
        z += "${nBox[i]} "
    }
    var kBox = ""
    for (i in 0..7) {
        val x = (nBox[i][0].toString() + nBox[i][5].toString()).toInt(2)
        val y = nBox[i].substring(1, 5).toInt(2)
        var a = sBox[i][x][y].toString(2)
        while (a.length < 4) {
            a = "0$a"
        }
        kBox += a
    }
    var lastBox = ""
    for (i in kBox.indices) {
        lastBox += kBox[P[i]-1]
    }
    return lastBox
}
//операция xor
private fun isk(x: String, y: String): String {
    var z = ""
    for (i in x.indices) {
        z += if (x[i] != y[i]) {
            "1"
        } else {
            "0"
        }
    }
    return z
}
//генерация раундовых ключей для шифрования
private fun keyGenForRound(key: String): Array<String> {
    var nKey = ""
    for (i in key.indices) {
        var a = key[i].hashCode().toString(2)
        while (a.length < 8) {
            a = "0$a"
        }
        nKey += a
    }
    var kl = ""
    for (i in 0..55) {
        kl += nKey[firstKeyInitial[i]-1]
    }
    val c = Array(17) { "" }
    val d = Array(17) { "" }

```

```

    val k = Array(16) {""}
    c[0] = kl.substring(0,28)
    d[0] = kl.substring(28)
    for(i in 1..16){
        c[i] = c[i-1].substring(rightShift[i-1])+c[i-1].substring(0,
rightShift[i-1])
        d[i] = d[i-1].substring(rightShift[i-1])+d[i-1].substring(0,
rightShift[i-1])
        for(j in 0..47){
            k[i-1] += (c[i]+d[i])[lastKeyInitial[j]-1].toString()
        }
    }
    return k
}
//первоначальная перестановка ключа Или же удаление проверочных битов ключа+перестановка
private val firstKeyInitial = arrayOf(
    57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4)
//конечная перестановка ключа или же перестановка сжатия ключа
private val lastKeyInitial = arrayOf(
    14, 17, 11, 24, 1, 5,
    3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8,
    16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55,
    30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53,
    46, 42, 50, 36, 29, 32)
//сдвиг right shift
private val rightShift = arrayOf(1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2,
1)
//первоначальная перестановка исходного текста
private val IP = arrayOf(
    58, 50, 42, 34, 26, 18, 10, 2,
    60, 52, 44, 36, 28, 20, 12, 4,
    62, 54, 46, 38, 30, 22, 14, 6,
    64, 56, 48, 40, 32, 24, 16, 8,
    57, 49, 41, 33, 25, 17, 9, 1,
    59, 51, 43, 35, 27, 19, 11, 3,
    61, 53, 45, 37, 29, 21, 13, 5,
    63, 55, 47, 39, 31, 23, 15, 7)
//окончательная перестановка исходного текста
private val IP_1 = arrayOf(
    40, 8, 48, 16, 56, 24, 64, 32,
    39, 7, 47, 15, 55, 23, 63, 31,
    38, 6, 46, 14, 54, 22, 62, 30,
    37, 5, 45, 13, 53, 21, 61, 29,
    36, 4, 44, 12, 52, 20, 60, 28,
    35, 3, 43, 11, 51, 19, 59, 27,
    34, 2, 42, 10, 50, 18, 58, 26,
    33, 1, 41, 9, 49, 17, 57, 25)
//расширение R в исходном тексте
private val e_R = arrayOf(
    32, 1, 2, 3, 4, 5,
    4, 5, 6, 7, 8, 9,
    8, 9, 10, 11, 12, 13,
    12, 13, 14, 15, 16, 17,
    16, 17, 18, 19, 20, 21,

```

```

20, 21, 22, 23, 24, 25,
24, 25, 26, 27, 28, 29,
28, 29, 30, 31, 32, 1
)
//sBox
private val sBox = arrayOf(
    arrayOf(
        arrayOf(14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7),
        arrayOf(0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8),
        arrayOf(4,1,14,8,13,6,2,11,15,12,9,7,13,10,5,0),
        arrayOf(15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13)
    ),
    arrayOf(
        arrayOf(15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10),
        arrayOf(3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5),
        arrayOf(0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15),
        arrayOf(13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9)
    ),
    arrayOf(
        arrayOf(10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8),
        arrayOf(13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1),
        arrayOf(13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7),
        arrayOf(1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12)
    ),
    arrayOf(
        arrayOf(7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15),
        arrayOf(13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9),
        arrayOf(10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4),
        arrayOf(3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14)
    ),
    arrayOf(
        arrayOf(2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9),
        arrayOf(14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6),
        arrayOf(4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14),
        arrayOf(11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3)
    ),
    arrayOf(
        arrayOf(12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11),
        arrayOf(10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8),
        arrayOf(9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6),
        arrayOf(4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13)
    ),
    arrayOf(
        arrayOf(4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1),
        arrayOf(13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6),
        arrayOf(1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2),
        arrayOf(6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12)
    ),
    arrayOf(
        arrayOf(13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7),
        arrayOf(1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2),
        arrayOf(7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8),
        arrayOf(2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11)
    )
)
//перестановка функции f
private val P = arrayOf(
    16, 7, 20, 21, 29, 12, 28, 17,
    1, 15, 23, 26, 5, 18, 31, 10,
    2, 8, 24, 14, 32, 27, 3, 9,
    19, 13, 30, 6, 22, 11, 4, 25
)

```

```

)
//Преобразование текста для шифрования
private fun textTransform(text: String, encOrDec: Int): Array<String> {
    var txt = ""
    if(encOrDec==1) {
        txt = text.base64Encode()
        if(txt.length%8!=0){
            repeat(8-txt.length%8){txt += " "}
        }
    }
    if(encOrDec==2) {
        txt = text.base64Decode()
    }
    val partedText = txt.partitioner(txt.length/8, 8)
    return partedText
}
//Преобразование ключа
private fun keyTransform(key: String): String {
    //    ограничить длину ключа максимум на 12 Анг или 8 кириллица
    var k = key.base64Encode()
    if(k.length>8){
        k = k.substring(0, 8)
    }else if(k.length<8){
        repeat(8-k.length%8){k = "0$k"}
    }
    return k
}
//partitioner extension function для деления текста на части с указанной длиной// Возвращает Array<String>
private fun String.partitioner(arraySize: Int, partLength: Int): Array<String> {
    return Array(arraySize) { i ->
        this.substring(i * partLength, (i + 1) * partLength)
    }
}
//extensions functions для кодировки и декодировки// Возвращает String
private fun String.base64Encode(): String { return
    Base64.getEncoder().encodeToString(this.toByteArray()) }
private fun String.base64Decode(): String { return
    String(Base64.getDecoder().decode(this)) }

```

ПРИЛОЖЕНИЕ Д

Алгоритм шифрования «Магма»

```
import java.util.*
private val sc = Scanner(System.`in`)
fun main(){
    println(magmaEncrypt())
    println(magmaDecrypt())
}
//enc function
fun magmaEncrypt(): String {
    val roundKeys = keyShuffler(keyTransform("ключ для магмы"),1)
    val txt = textTransform("текст для магмы", 1)
    var endText = ""
    for (block in txt.indices){
        var l = txt[block].substring(0, 8).toULong(16)
        var r = txt[block].substring(8).toULong(16)
        var changeBox: ULong
        repeat(31){
            changeBox = l
            l = r
            r = changeBox xor f(r, roundKeys[it], block, it)
        }
        var x = (l xor f(r, roundKeys[31], block, 31)).toString(16)
        var y = r.toString(16)
        if(x.length<8){
            repeat(8-x.length%8){x = "0$x"}
        }
        if(y.length<8){
            repeat(8-y.length%8){y = "0$y"}
        }
        endText += (x+y)
    }
    var encText = ""
    val xyz = endText.partitioner(endText.length/2, 2)
    repeat(xyz.size){ encText += xyz[it].toInt(16).toChar() }
    return encText.base64Encode()
}
//dec function
fun magmaDecrypt(): String {
    val roundKeys = keyShuffler(keyTransform("ключ для магмы"), 2).reversed()
    val txt = textTrans-
form("w5zDicOQwqh7bR8OI8KITVBDw5ZXaSDDlsKKw6B+EcK2wq8Cw5bCpGzDmSnCtcKCATlZQMK
rASvDlg==", 2)
    var endText = ""
    for (block in txt.indices){
        var l = txt[block].substring(0, 8).toULong(16)
        var r = txt[block].substring(8).toULong(16)
        var changeBox: ULong
        repeat(31){
            changeBox = l
            l = r
            r = changeBox xor f(r, roundKeys[it], block, it)
        }
        var x = (l xor f(r, roundKeys[31], block, 31)).toString(16)
        var y = r.toString(16)
        if(x.length<8){
            repeat(8-x.length%8){x = "0$x"}
        }
    }
```

```

    }
    if(y.length<8){repeat(8-y.length%8){y = "0$y"} }
    endText += (x+y)
}
var decText = ""
val xyz = endText.partitioner(endText.length/2, 2)
repeat(xyz.size){ decText += xyz[it].toInt(16).toChar() }
decText = decText.trim()
return decText.base64Decode()
}
//Преобразование текста для шифрования
private fun textTransform(text: String, endOrDec: Int): Array<String> {
    var txt = ""
    if(endOrDec==1) {
        txt = text.base64Encode()
        if(txt.length%8!=0){
            repeat(8-txt.length%8){txt += " "}
        }
    }
    if(endOrDec==2) {
        txt = text.base64Decode()
    }
    val partedText = txt.partitioner(txt.length/8, 8)
    val endText = Array(partedText.size){""}
    var x: ULong
    for(i in endText.indices){
        x = 0UL
        for(j in partedText[i].indices){
            x = (x shl 8) xor partedText[i][j].code.toULong()
        }
        var temp =x.toString(16)
        if(temp.length<16){
            repeat(16-temp.length%16){temp = "0$temp"}
        }
        endText[i] += temp
    }
    return endText
}
//Преобразование ключа
private fun keyTransform(key: String): String {
    // ограничить длину ключа максимум на 24 АнГ или 12 кириллица
    var k = key.base64Encode()
    if(k.length>32){
        k = k.substring(0, 32)
    }else if(k.length<32){
        repeat(32-k.length%32){k = "0$k"}
    }
    var endKeyValue = ""
    return endKeyValue
}
//Развертывание ключей
private fun keyShuffler(key: String, encOrDec: Int): Array<ULong> {
    val k = key.partitioner(8, 8)
    val keys = Array(8){0UL}
    repeat(8){ keys[it] = k[it].toULong(16) }
    val roundKeysShuffle = Array(32){0UL}
    for(i in 0..31){
        if(i<24){
            roundKeysShuffle[i] = keys[i%8]
        }else{
            roundKeysShuffle[i] = keys[31-i]
        }
    }
}

```



```

    }
    }
    return roundKeysShuffle
}
//вызываются все остальные функции, а также производится сумма по модулю 2 в
степени 32
private fun f(x: ULong, k: ULong, block: Int, i: Int): ULong {
    val xyz: ULong = sBoxReplace((x+k)%(1UL shl 32), block, i)
    return rot11(xyz, block, i)
}
//циклический сдвиг на 11 битов
private fun rot11(x: ULong, block: Int, i: Int): ULong{
    if(block == 0 && i == 0){
        var z = x.toString(2)
        if(z.length<32){
            z = "0$z"
        }
        z = (((x shl 11) xor (x shr 21)) and mask32).toString(2)
        if(z.length<32){
            z = "0$z"
        }
    }
    return ((x shl 11) xor (x shr 21)) and mask32
}
//перестановка S_Box
private fun sBoxReplace(x: ULong, block: Int, xyz: Int): ULong {
    var y: ULong = 0U
    for (i in 7 downTo 0) {
        val j = (x shr (4 * i)).toInt() and 0xf
        y = y shl 4
        y = y xor s_box[7-i][j].toULong()
    }
    return y
}
//box
private val s_box = arrayOf (
    arrayOf(12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1),
    arrayOf(6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 1, 13, 0, 15),
    arrayOf(11, 3, 5, 8, 2, 15, 10, 13, 14, 1, 7, 4, 12, 9, 6, 0),
    arrayOf(12, 8, 2, 1, 13, 4, 15, 6, 7, 0, 10, 5, 3, 14, 9, 11),
    arrayOf(7, 15, 5, 10, 8, 1, 6, 13, 0, 9, 3, 14, 11, 4, 2, 12),
    arrayOf(5, 13, 15, 6, 9, 2, 12, 10, 11, 7, 8, 1, 4, 3, 14, 0),
    arrayOf(8, 14, 2, 5, 6, 9, 1, 12, 15, 4, 11, 0, 13, 10, 3, 7),
    arrayOf(1, 7, 14, 13, 0, 5, 8, 3, 4, 15, 10, 6, 9, 12, 11, 2)
)
//маска для 32 бита
private val mask32 = UInt.MAX_VALUE.toULong()
//partitioner extension function для деления текста на части с указанной дли-
ной// Возвращает Array<String>
private fun String.partitioner(arraySize: Int, partLength: Int): Ar-
ray<String> {
    return Array(arraySize) { i ->
        this.substring(i * partLength, (i + 1) * partLength)
    }
}
//extensions functions для кодировки и декодировки// Возвращает String
private fun String.base64Encode(): String { return
    Base64.getEncoder().encodeToString(this.toByteArray()) }
private fun String.base64Decode(): String { return
    String(Base64.getDecoder().decode(this)) }

```

ПРИЛОЖЕНИЕ Ж

Алгоритм шифрования «RSA»

```
package com.company
import java.math.BigInteger
import java.security.SecureRandom
import java.util.*
private var e = 0
private var d = 0
private var mod = 0
private val sc = Scanner(System.`in`)
fun main() {
    println(genKeyForRSA())
    println(encryptRSA())
    println(decryptRSA())
}
//Шифрование
fun encryptRSA(): String {
    val text = sc.nextLine()
    var encText = ""
    for (i in text.indices) {
        encText +=
            (text[i].hashCode().toBigInteger().pow(e).rem(mod.toBigInteger()).toInt().toChar())
    }
    return encText
}
//дешифрование
fun decryptRSA(): String {
    val text = sc.nextLine()
    val tx= text.base64Decode()
    var decText = ""
    for (i in tx.indices) {
        decText +=
            (tx[i].hashCode().toBigInteger().pow(d).rem(mod.toBigInteger()).toInt().toChar())
    }
    return decText
}
//генерация ключей
fun genKeyForRSA() {
    val primeP = generatePrime(2,200)
    val primeQ = generatePrime(2,200)
    mod = primeP*primeQ
    val f = (primeP-1)*(primeQ-1)
    e = generatePrime(2,f)
    while (true){
        if(f%e==0){
            e = generatePrime(2,f)
        }
        else{
            break
        }
    }
    var i = 2
    while (true){
        if((f*i+1)%e==0){
            d = (f*i+1)/e
        }
    }
}
```

```

        if (d!=e) {
            break
        }
    }
    i++
}
}
//генерация простого числа от otX до doX
fun generatePrime(x: Int, y: Int): Int {
    require(x < y) { "x должен быть меньше y" }
    require(x > 1) { "x должен быть больше 1" }
    val random = SecureRandom()
    while (true) {
        val n = random.nextInt(y - x) + x
        if (BigInteger.valueOf(n.toLong()).isProbablePrime(100)) {
            return n
        }
    }
}
}
//extensions functions для кодировки и декодировки// Возвращает String
private fun String.base64Encode(): String { return
Base64.getEncoder().encodeToString(this.toByteArray()) }
private fun String.base64Decode(): String { return
String(Base64.getDecoder().decode(this)) }

```

ПРИЛОЖЕНИЕ И

Алгоритм шифрования «Схема Эль-Гамала»

```
import java.math.BigInteger
import java.security.SecureRandom
import java.util.*
private var publicPrimeP = 0
private var publicRandomG = 0
private var publicValueA = 0
private var publicValueY = 0
private var privateRandomX = 0
private val sc = Scanner(System.`in`)
fun main() {
    genKeyForElGamal()
    encryptElGamal()
    decryptElGamal()
}
fun decryptElGamal(): String {
    val text = sc.nextLine()
    val tx = text.base64Decode()
    var decText = ""
    for (i in tx.indices) {
        val resX = publicValueA.toBigInteger().pow(publicPrimeP-1-
privateRandomX)
        val resY = resX.multiply(tx[i].code.toBigInteger())
        decText += ( resY ).rem(publicPrimeP.toBigInteger()).toInt().toChar()
    }
    return decText
}
fun encryptElGamal(): String {
    val text = "привет kiFe"
    var encText = ""
    val sessionKey = (2..< (publicPrimeP-1)).random()
    publicValueA = ( publicRandomG.toBigInteger().modPow(sessionKey.toBigInteger(), publicPrimeP.toBigInteger()) ).toInt()
    val resY = publicRandomG.toBigInteger().pow(sessionKey).toString()
    for (i in text.indices) {
        val resX = publicValueY.toBigInteger().pow(sessionKey)
        val resY = resX.multiply(text[i].code.toBigInteger())
        encText += ( resY ).rem(publicPrimeP.toBigInteger()).toInt().toChar()
    }
    return encText.base64Encode()
}
fun genKeyForElGamal() {
    publicPrimeP = generatePrime(1500, 2000)
    publicRandomG = (2..< publicPrimeP).random()
    privateRandomX = (2..< publicPrimeP).random()
    while (true) {
        if (privateRandomX == publicRandomG) {
            privateRandomX = (2..< publicPrimeP).random()
        }
        else {
            break
        }
    }
    publicValueY = ( publicRandomG.toBigInteger().modPow(privateRandomX.toBigInteger(), publicPrimeP.toBigInteger()) ).toInt()
}
```

```

        licPrimeP.toBigInteger()) ).toInt()
    }
    //генерация простого числа от otX до doX
    private fun generatePrime(x: Int, y: Int): Int {
        require(x < y) { "x должен быть меньше y" }
        require(x > 1) { "x должен быть больше 1" }
        val random = SecureRandom()
        while (true) {
            val n = random.nextInt(y - x) + x
            if (BigInteger.valueOf(n.toLong()).isProbablePrime(100)) {
                return n
            }
        }
    }
}
//extensions functions для кодировки и декодировки// Возвращает String
private fun String.base64Encode(): String { return
Base64.getEncoder().encodeToString(this.toByteArray()) }
private fun String.base64Decode(): String { return
String(Base64.getDecoder().decode(this)) }

```

ПРИЛОЖЕНИЕ К

Алгоритм обмена ключами «Протокол обмена ключами Диффи-Хеллмана»

```
import java.math.BigInteger
import java.security.SecureRandom
import java.util.*
private val sc = Scanner(System.`in`)
//основной метод
fun main() {
    println(diffieHellman())
}
fun diffieHellman() {
    val p = generatePrime(50,1000)
    val g = generatePrime(50,1000)
    val a = sc.nextInt()
    val b = sc.nextInt()
    val ga = g.toBigInteger().pow(a)
    val alisaPubKey = ga.rem(p.toBigInteger())
    val gb = g.toBigInteger().pow(b)
    val bobPubKey = gb.rem(p.toBigInteger())
    val ba = bobPubKey.pow(a)
    val bap = ba.rem(p.toBigInteger())
    val ab = alisaPubKey.pow(b)
    val abp = ab.rem(p.toBigInteger())
    println("$bap\t$abp")
}
//генерация простого числа от otX до doX
fun generatePrime(x: Int, y: Int): Int {
    require(x < y) { "x должен быть меньше y" }
    require(x > 1) { "x должен быть больше 1" }
    val random = SecureRandom()
    while (true) {
        val n = random.nextInt(y - x) + x
        if (BigInteger.valueOf(n.toLong()).isProbablePrime(100)) {
            return n
        }
    }
}
```